Docker Final Project

End-to-End Containerisation, Image Management & Custom Deployments on AWS EC2

Module 3: Docker Part 1 — Consolidated Assignments 1 to 5

Name: Rahul Kumar Pandey
Email: rahulpandeyyy@gmail.com
Mobile: 7257000333

1. Project Overview

This document merges five Docker lab assignments into a single end-to-end reference guide. Starting from a raw AWS EC2 Ubuntu instance, Docker is installed and a series of progressively advanced container workflows are demonstrated: pulling and running containers with port mapping, installing Apache2 inside a running container, saving a custom image, pushing it to Docker Hub, pulling it on a second machine, writing a Dockerfile to automate the build, and finally replacing the default Apache page with a custom HTML file baked into the image at build time.

Every screenshot is included in the order it was captured, with a comment below each one explaining the command that was run and what the output demonstrates.

1.1 Assignment Map

- Assignment 1 — Pull the Ubuntu image, run it with port 80 mapped, install Apache2 inside the container, and verify it in a browser.
- Assignment 2 — Commit the running container as a new image ('new-task'), launch it on port 81, and verify Apache2 in a browser.
- Assignment 3 — Tag and push the image to Docker Hub, provision a second EC2 instance (docker2), pull the image there, run it on port 80, and verify it.
- Assignment 4 — Write a Dockerfile that installs Apache2 and starts it automatically via ENTRYPOINT; build and run without manual steps.
- Assignment 5 — Extend the Dockerfile with an ADD instruction to replace the default Apache page with a custom HTML file; rebuild and verify in a browser.

2. Assignment 1 — Pull Ubuntu Container & Install Apache2

Tasks: (1) Pull the Ubuntu container image. (2) Run it with port 80 mapped to the host. (3) Install Apache2 inside the container. (4) Verify the Apache page in a browser.

2.1 Deployment Steps

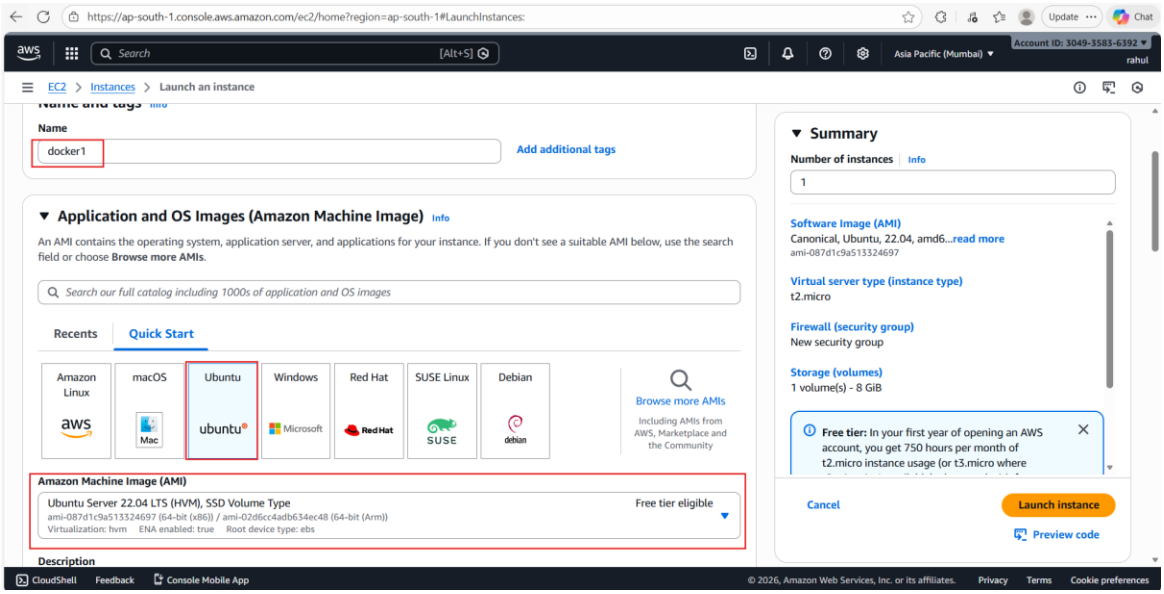
On the EC2 Host (docker1):

```
sudo apt update
sudo apt install docker.io -y
docker --version
sudo docker pull ubuntu
sudo docker images
sudo docker run -itd -p 80:80 ubuntu
sudo docker ps
sudo docker exec -it c6c2ab2c017a bash
```

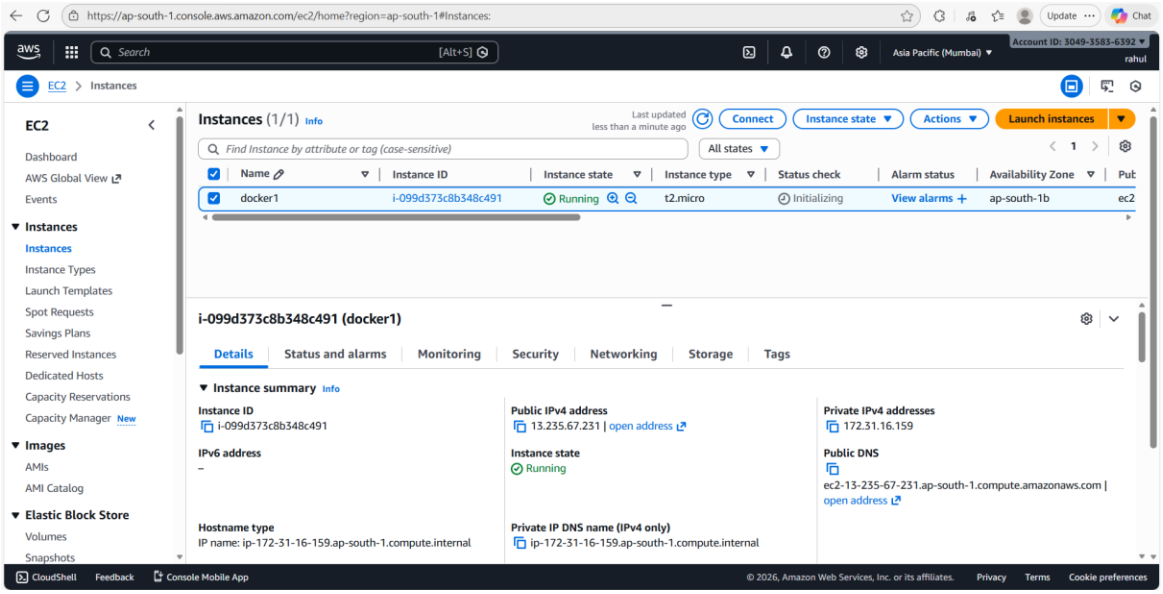
Inside the Container:

```
apt-get update
apt install apache2 -y
service apache2 status
service apache2 start
service apache2 status
```

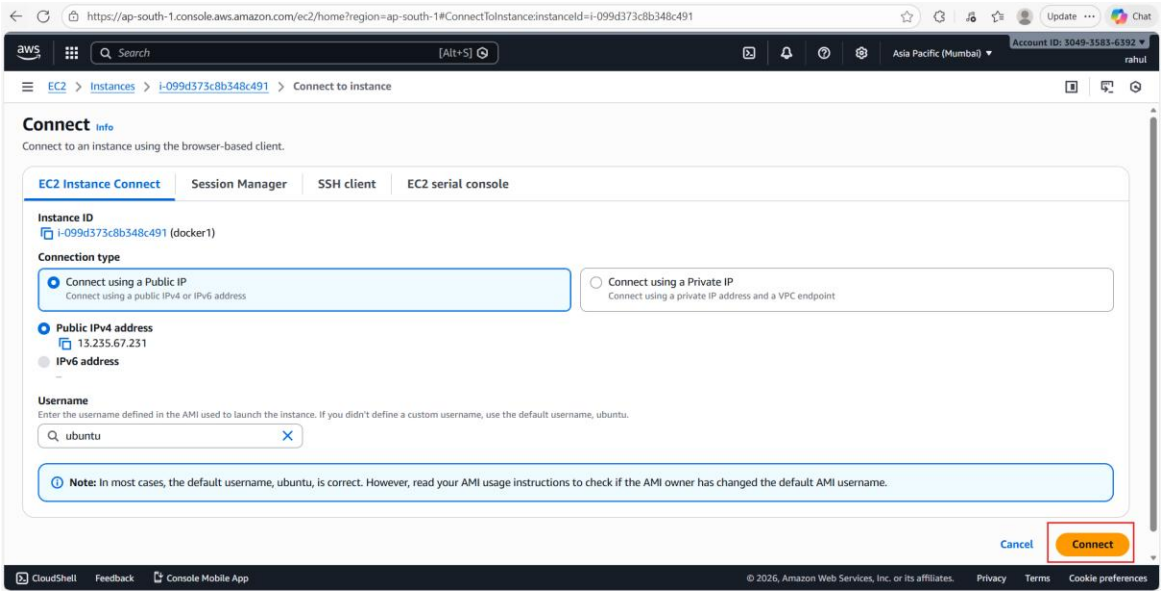
2.2 Screenshots & Comments



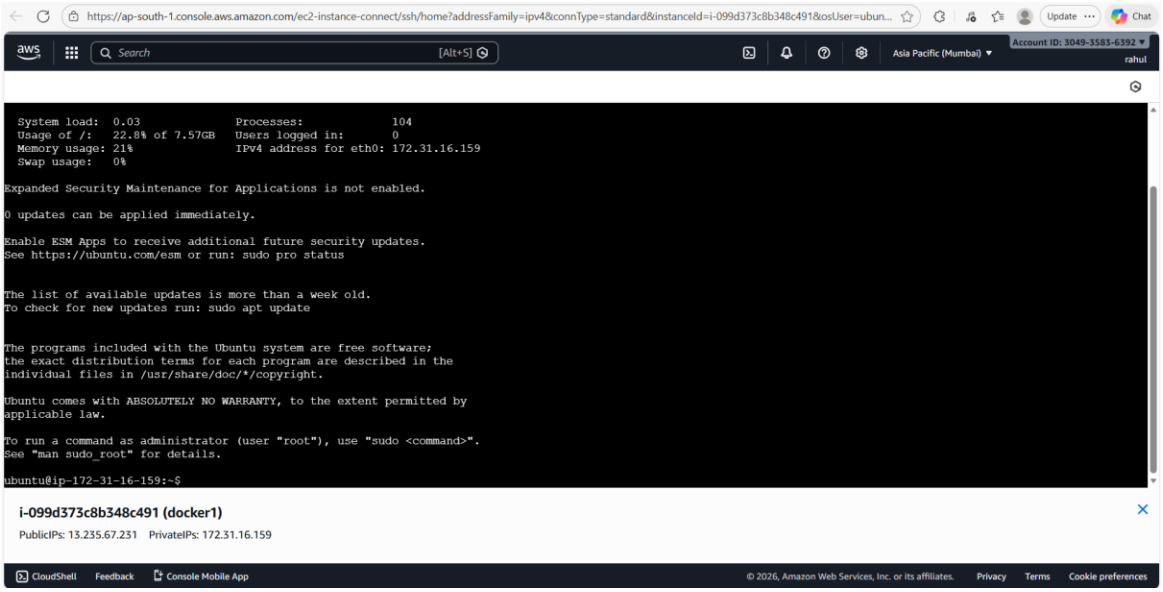
Launching a single EC2 instance named 'docker1' on Ubuntu 22.04 LTS (t2.micro, ap-south-1 region) from the AWS console — this will be the Docker host for Assignments 1–2.



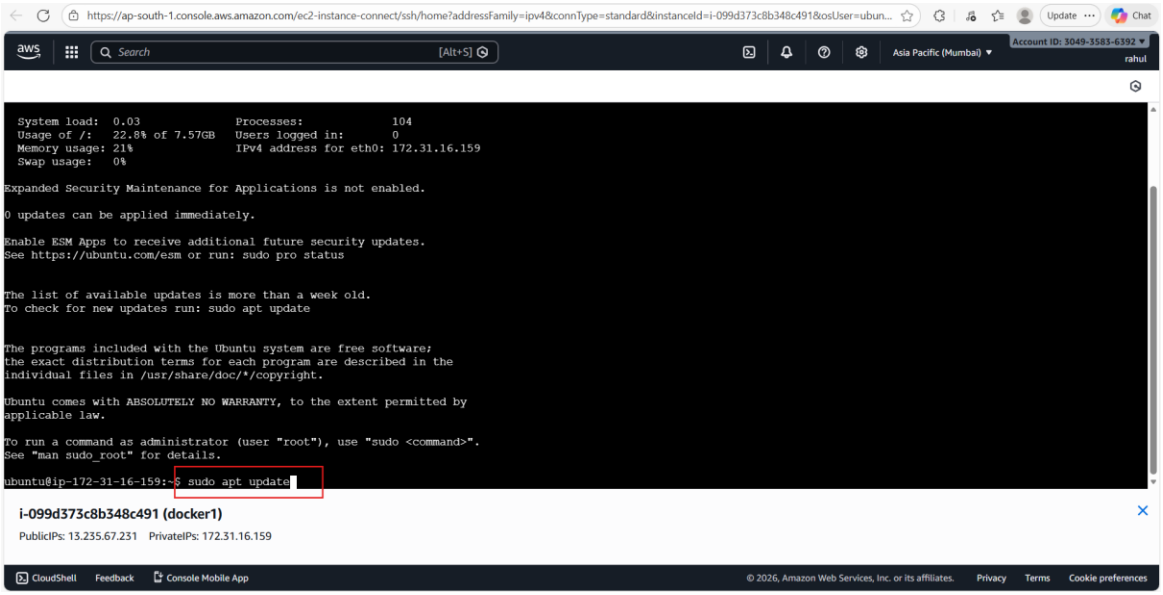
The docker1 instance is Running with Public IP 13.235.67.231 and Private IP 172.31.16.159, ready to connect.



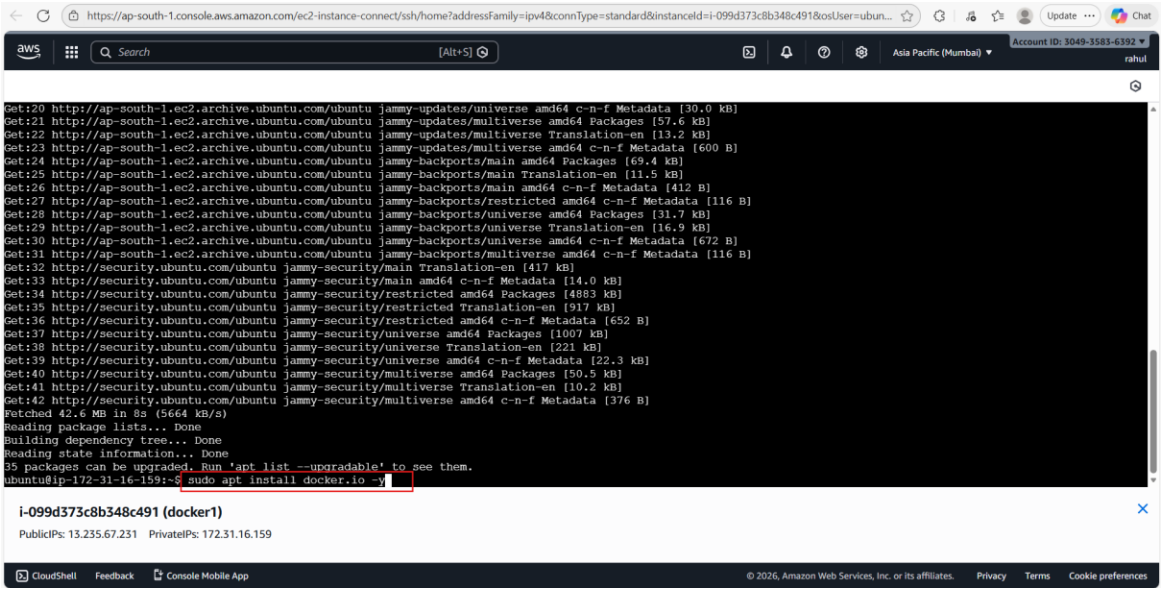
Connecting to docker1 via EC2 Instance Connect using the public IPv4 address and default ubuntu username — clicking the 'Connect' button opens a browser-based SSH shell.



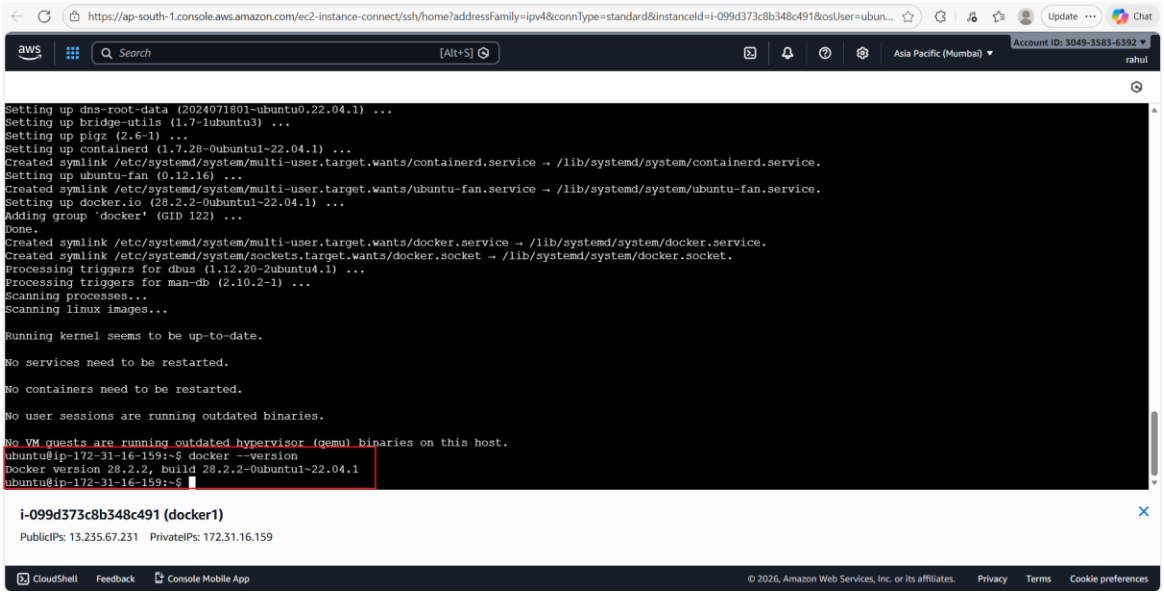
Landed inside docker1's terminal. The system shows low load (0.03) and 22.8% disk usage. The prompt is ready for commands.



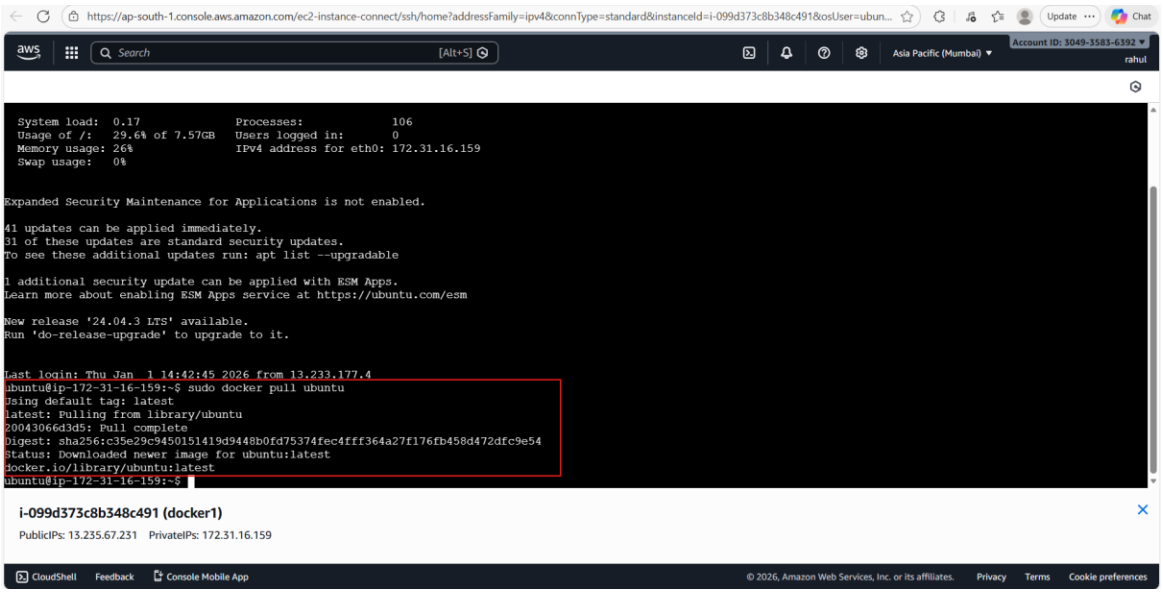
Running 'sudo apt update' to refresh the package lists before installing Docker.



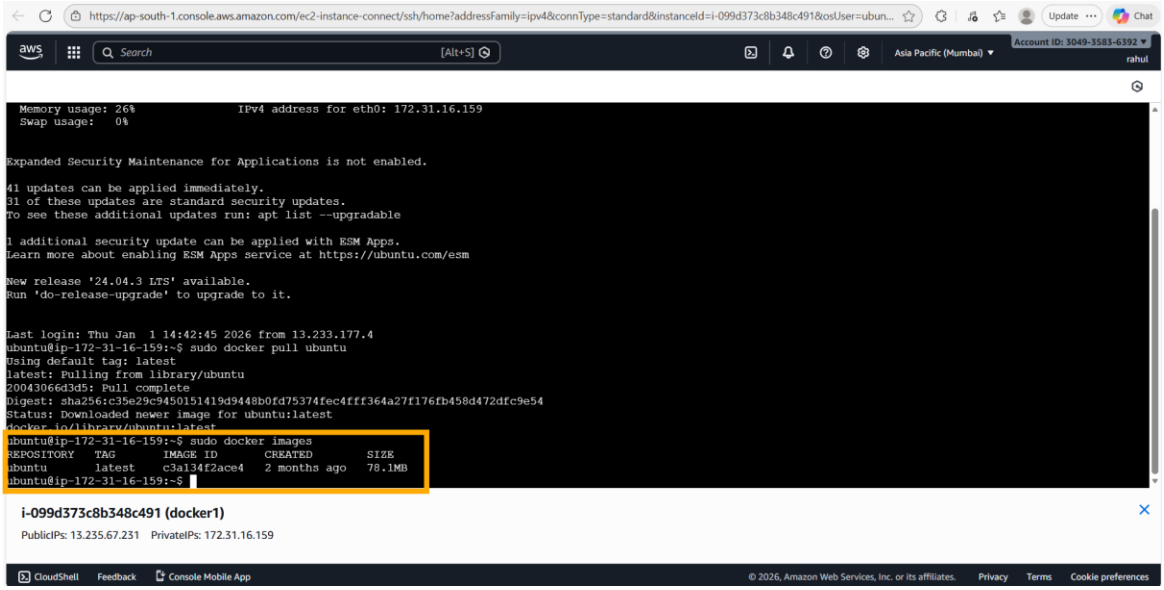
Package lists updated (42.6 MB fetched); now running 'sudo apt install docker.io -y' to install the Docker engine.



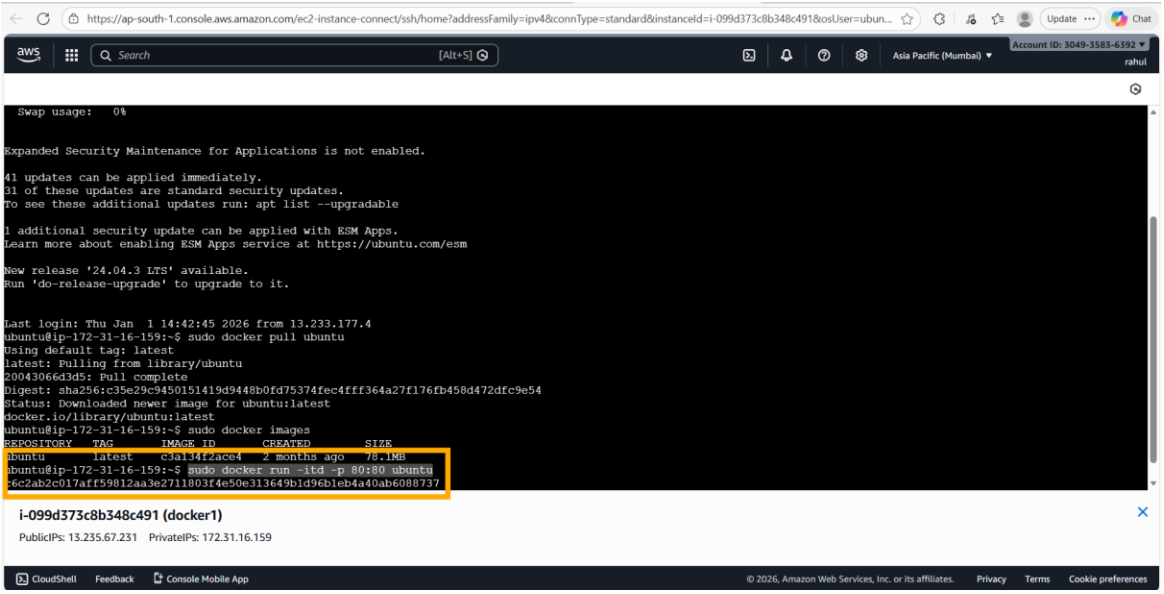
'docker --version' confirms Docker 28.2.2 is installed and the docker.service symlink is active.



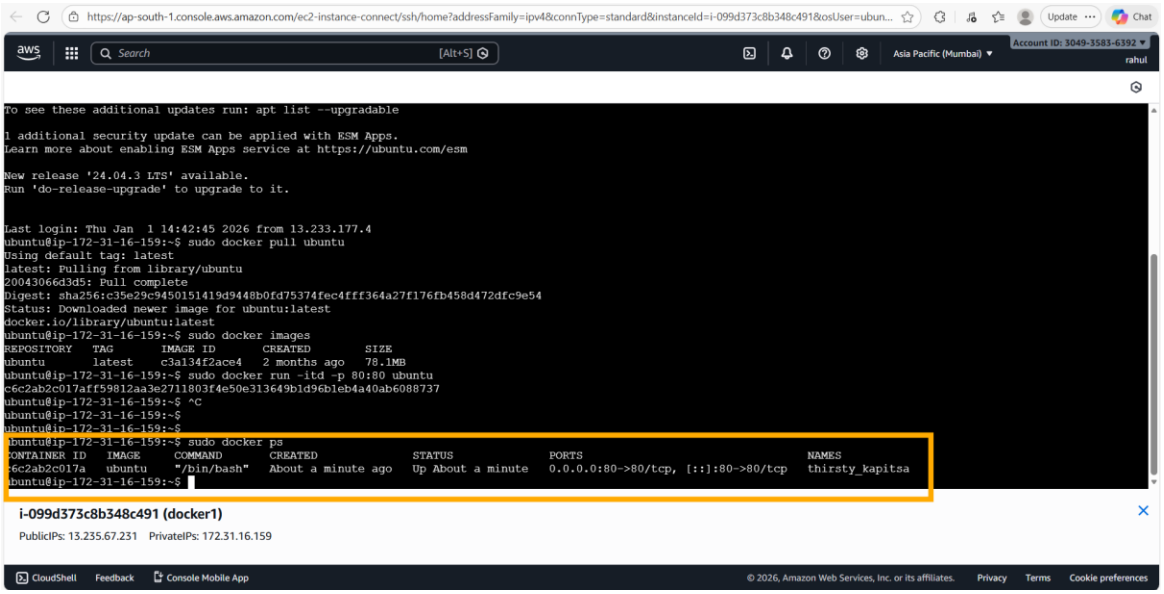
'sudo docker pull ubuntu' downloads the latest Ubuntu image from Docker Hub. The pull completes and the image ID is shown in the confirmation line.



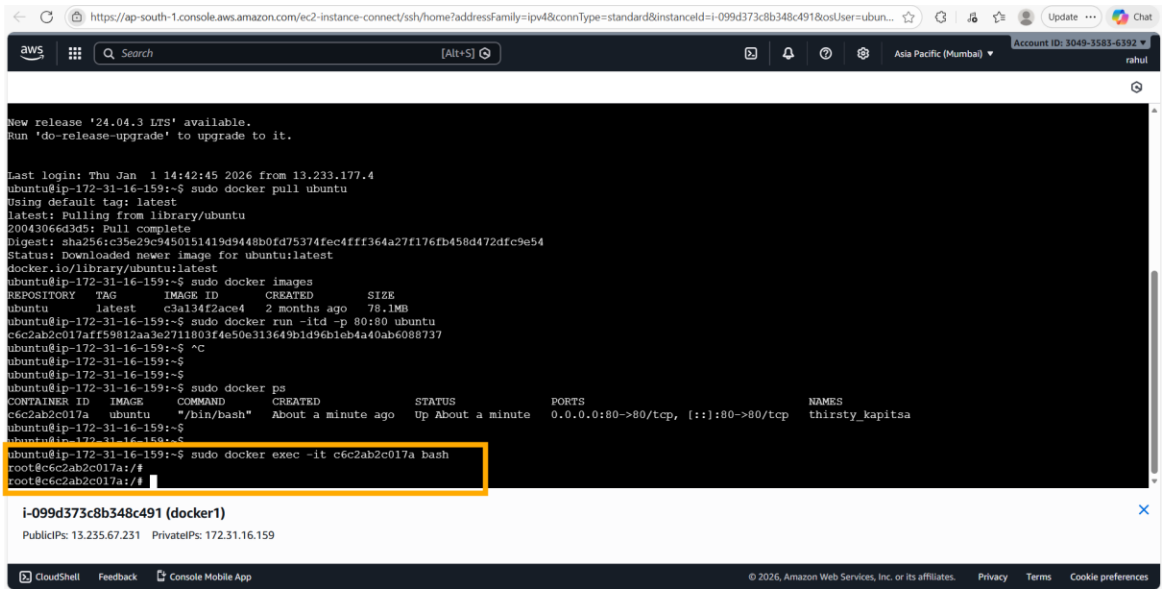
'sudo docker images' lists the downloaded ubuntu:latest image (78.1 MB, created 2 months ago). Next, 'sudo docker run -itd -p 80:80 ubuntu' launches the container in detached interactive mode, mapping host port 80 to container port 80 — the full container ID hash is printed.



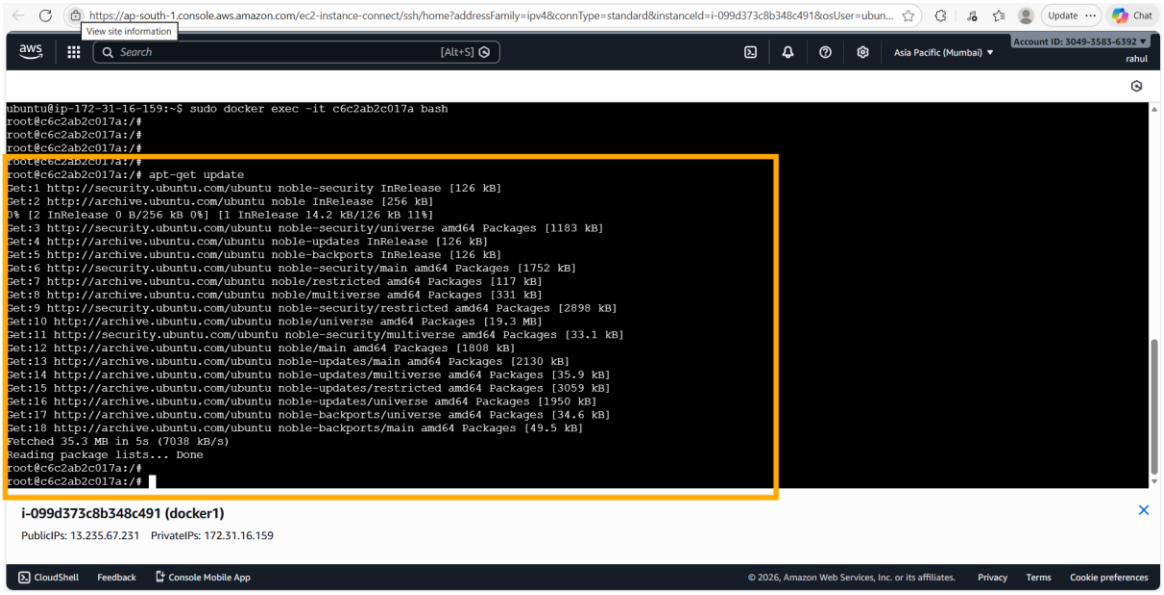
Running 'sudo docker run -itd -p 80:80 ubuntu' — the container hash (c6c2ab2c017a…) is returned and the container starts in the background.



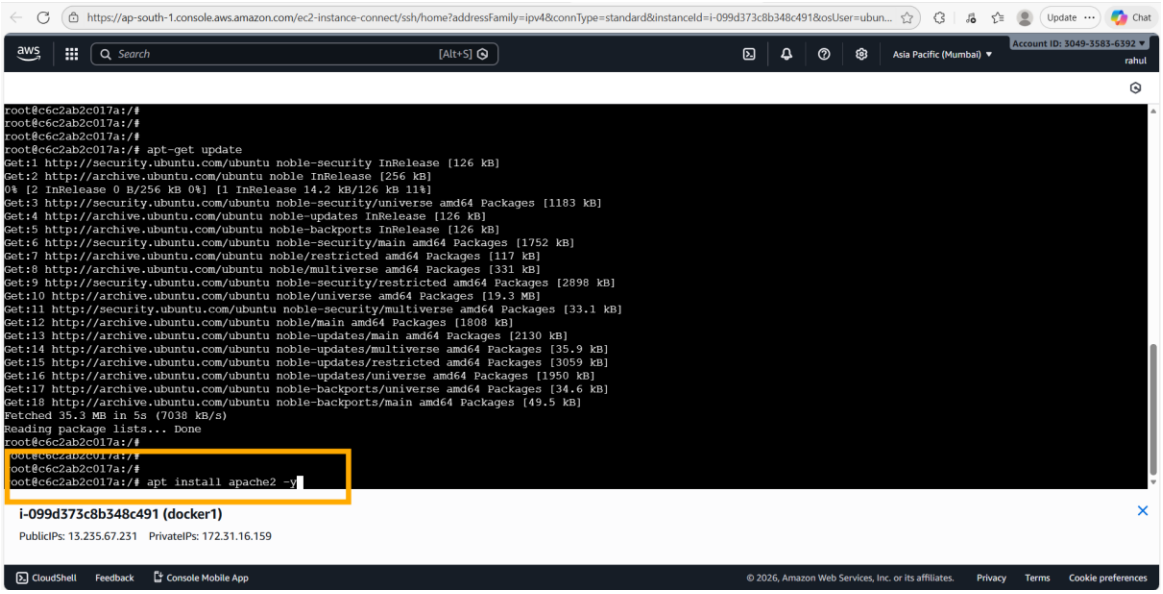
'sudo docker ps' confirms container c6c2ab2c017a is Up and listening on 0.0.0.0:80->80/tcp (named 'thirsty_kapitsa'). Then 'sudo docker exec -it c6c2ab2c017a bash' opens an interactive bash shell inside the running container.



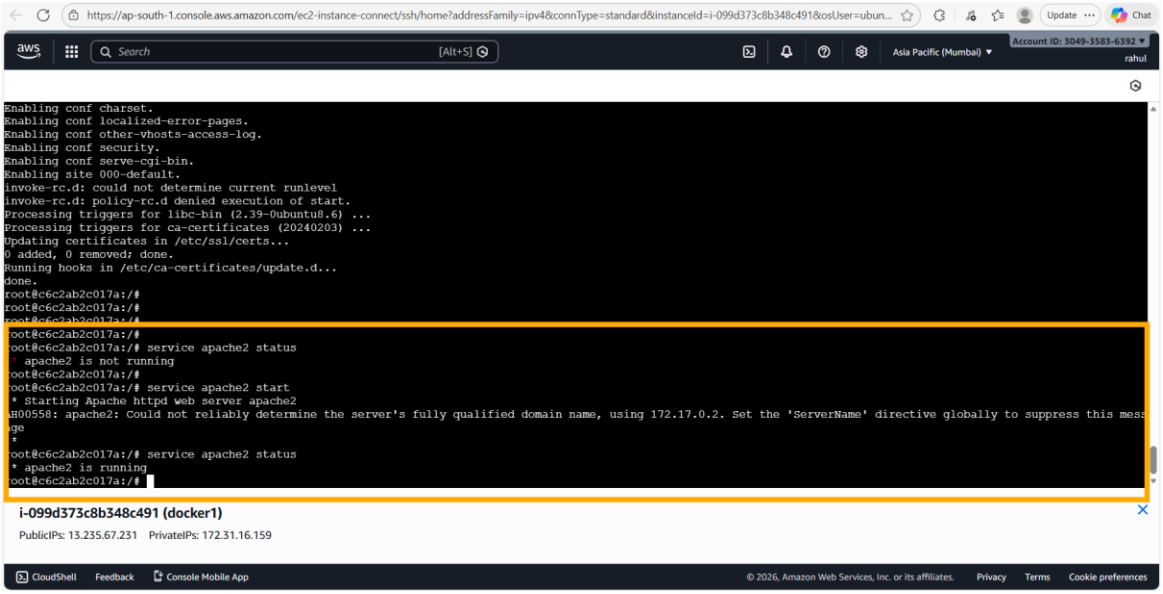
Shell is now inside the container (root@c6c2ab2c017a:/#). Running 'apt-get update' to refresh package lists inside the Ubuntu container.



'apt-get update' inside the container fetches 35.3 MB of package metadata from Ubuntu's Noble (24.04) repositories.



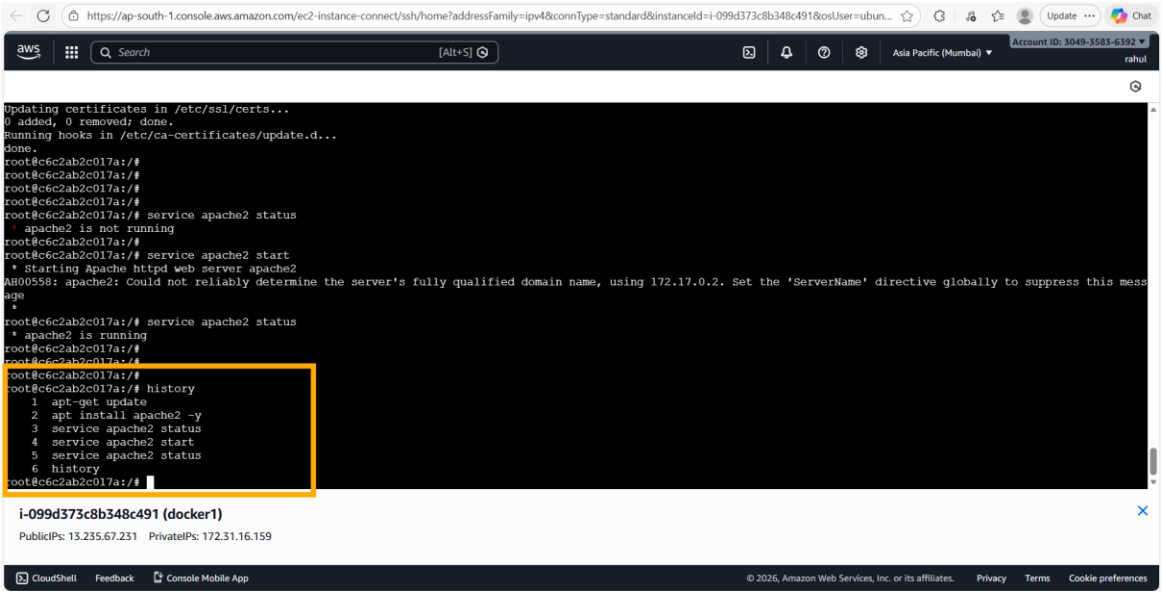
Running 'apt install apache2 -y' inside the container — Apache2 and all its dependencies are being installed.



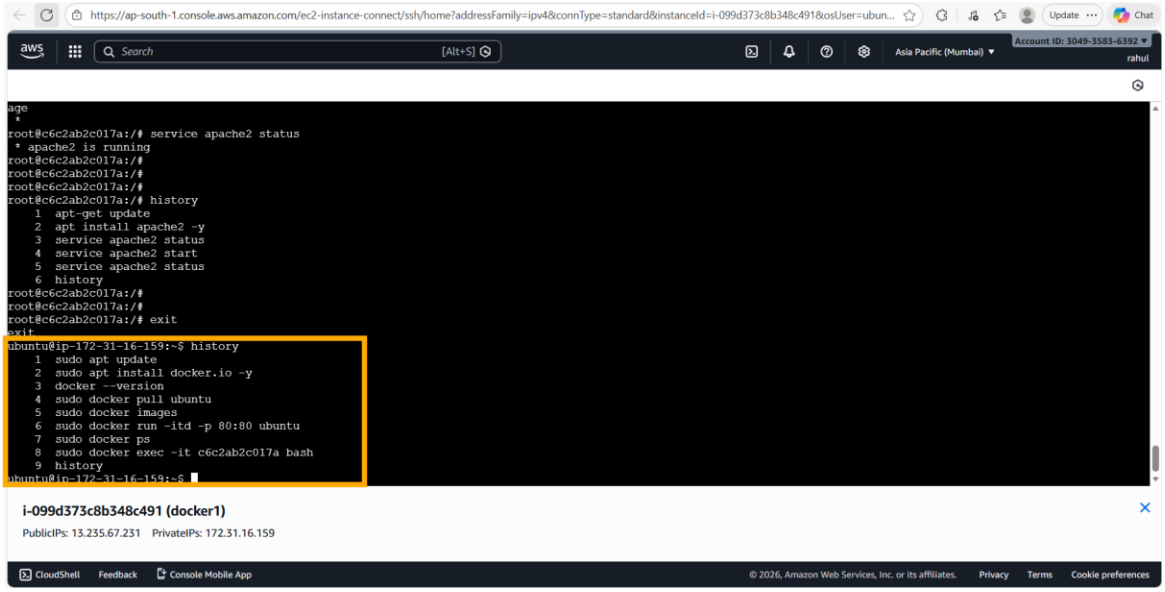
Apache2 installed. 'service apache2 status' shows it is NOT running initially. 'service apache2 start' starts it (minor ServerName warning is harmless). 'service apache2 status' then confirms ' apache2 is running'.*



Browser verification: navigating to 13.235.67.231 (port 80) shows the Ubuntu Apache2 Default Page with 'It works!' — proving the container's Apache2 is reachable from outside through the -p 80:80 port mapping.



Container command history: apt-get update → apt install apache2 -y → service apache2 status → service apache2 start → service apache2 status. Host history shows the full Docker command sequence used.



Full host command history: 1) sudo apt update 2) sudo apt install docker.io -y 3) docker --version 4) sudo docker pull ubuntu 5) sudo docker images 6) sudo docker run -itd -p 80:80 ubuntu 7) sudo docker ps 8) sudo docker exec -it c6c2ab2c017a bash — confirms the complete workflow.

3. Assignment 2 — Commit Image & Run on Port 81

Tasks: (1) Save the running container from Assignment 1 as a new Docker image. (2) Launch a container from this new image mapped to port 81. (3) Start Apache2 inside the new container. (4) Verify in a browser.

3.1 Deployment Steps

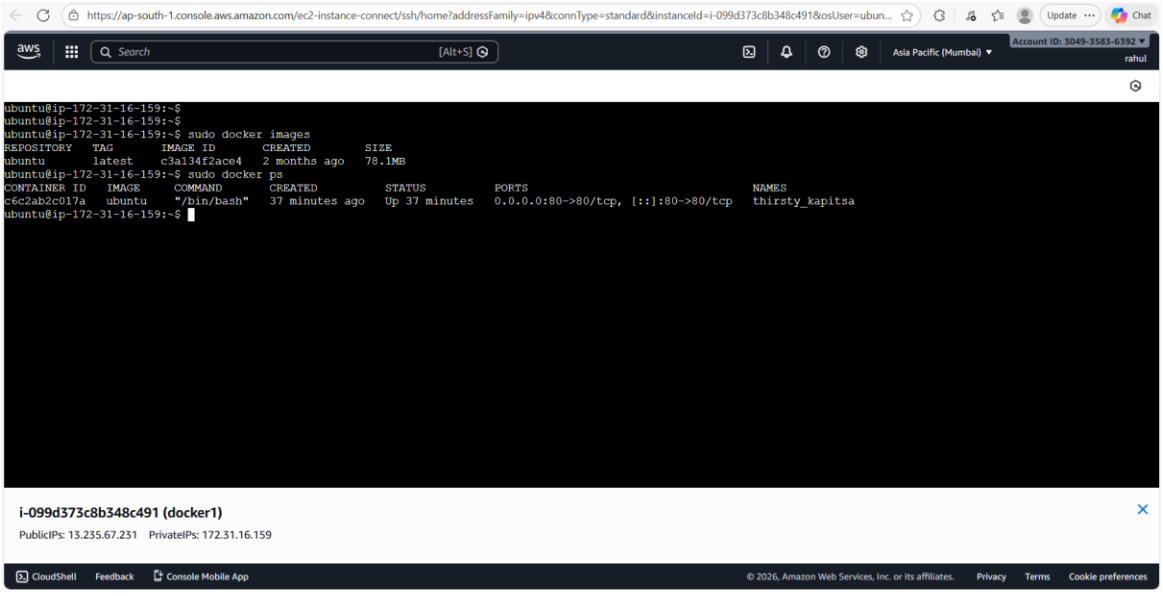
On the EC2 Host (docker1):

```
sudo docker images
sudo docker ps
sudo docker commit c6c2ab2c017a new-task
sudo docker images
sudo docker run -itd -p 81:80 new-task
sudo docker ps
sudo docker exec -it 1d2e7438afd8 bash
```

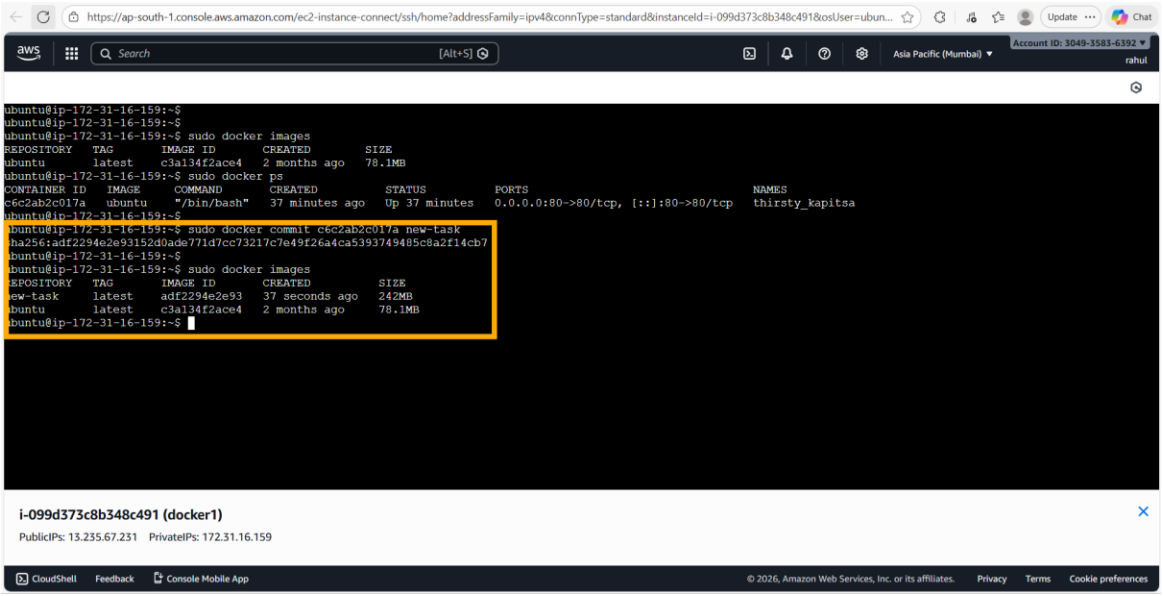
Inside the New Container:

```
service apache2 status
service apache2 start
service apache2 status
```

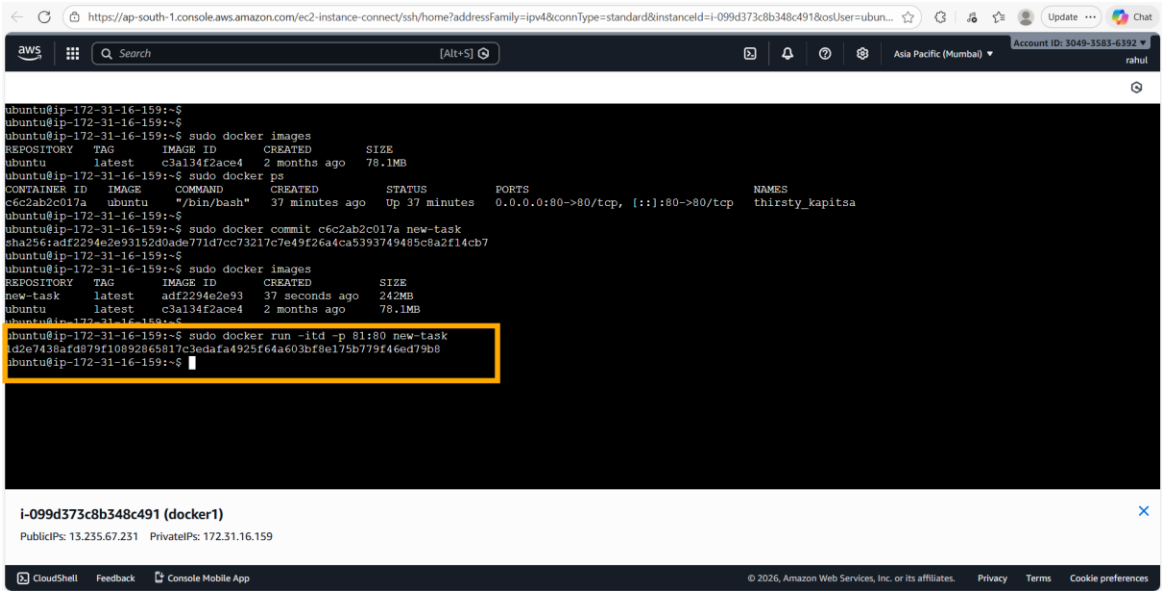
3.2 Screenshots & Comments



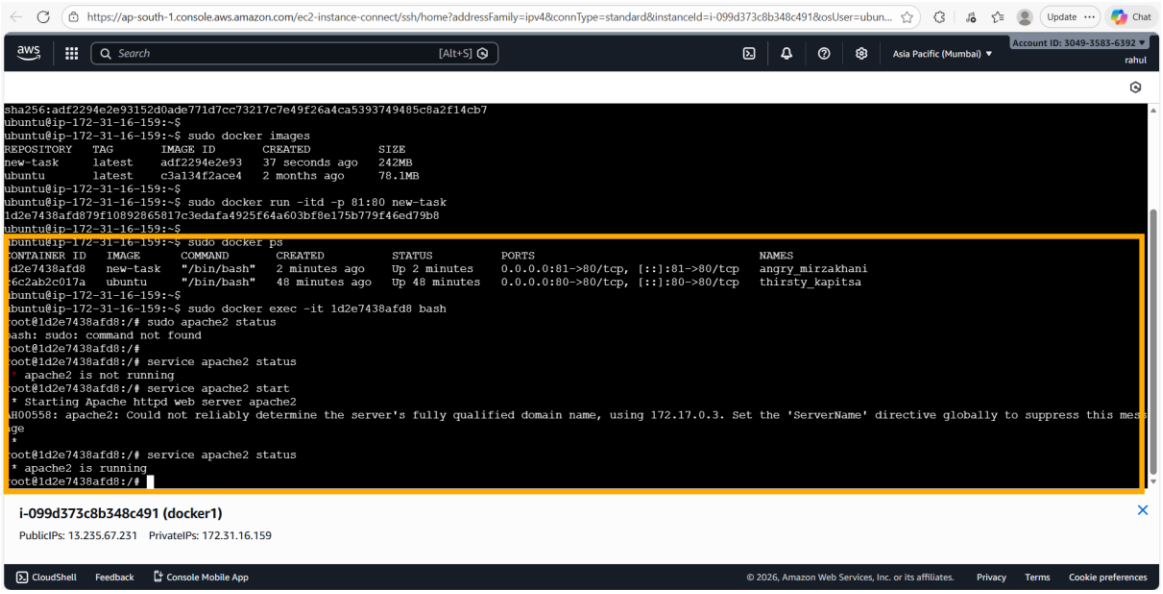
'sudo docker images' lists the ubuntu:latest base image. 'sudo docker ps' confirms the original container (c6c2ab2c017a) is still running on port 80:80.



'sudo docker commit c6c2ab2c017a new-task' creates a snapshot of the live container's filesystem as a new image named 'new-task' (SHA256: adf2294e2e93, 242 MB). 'sudo docker images' now shows both ubuntu:latest and new-task:latest side by side.



'sudo docker run -itd -p 81:80 new-task' launches a second container from the committed image, mapping host port 81 to container port 80. The new container ID (1d2e7438afd8…) is printed.



'sudo docker ps' shows both containers running simultaneously: c6c2ab2c017a (ubuntu, port 80) and 1d2e7438afd8 (new-task, port 81). Inside the new container, 'service apache2 status' shows it is not running (the committed state didn't include a running service), so 'service apache2 start' is run and the final status confirms ' apache2 is running'.*



Browser verification: navigating to 13.235.67.231:81 shows the Apache2 Default Page — confirming that the committed 'new-task' image retains the Apache2 installation and the container serves traffic on port 81.

4. Assignment 3 — Push to Docker Hub & Pull on a Second Machine

Tasks: (1) Log in to Docker Hub. (2) Tag the 'new-task' image with the Docker Hub username. (3) Push the image. (4) Launch a second EC2 instance (docker2), install Docker, pull the image, run it on port 80, start Apache2, and verify in a browser.

4.1 Deployment Steps

On docker1 (push):

```
sudo docker images
sudo docker login
# Complete web-based device activation at https://login.docker.com/activate
sudo docker tag new-task rahulpandeyyy/new-task
sudo docker images
sudo docker push rahulpandeyyy/new-task
```

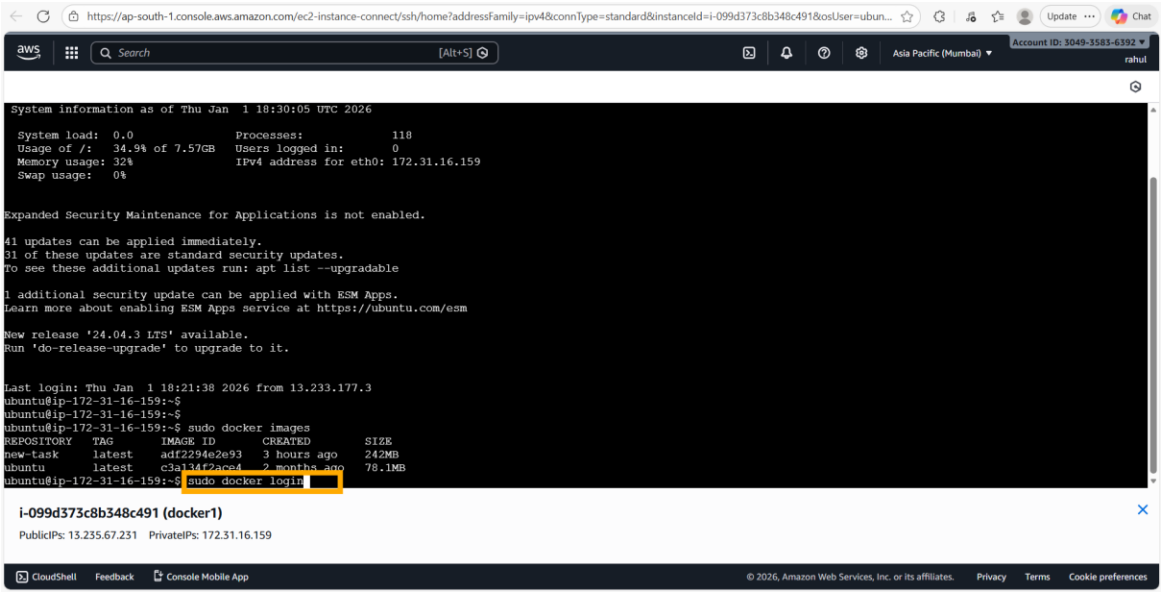
On docker2 (pull & run):

```
sudo apt update
sudo apt install docker.io -y
docker --version
sudo docker pull rahulpandeyyy/new-task
sudo docker images
sudo docker run -itd -p 80:80 rahulpandeyyy/new-task
sudo docker ps
sudo docker exec -it 420111ce9c31 bash
```

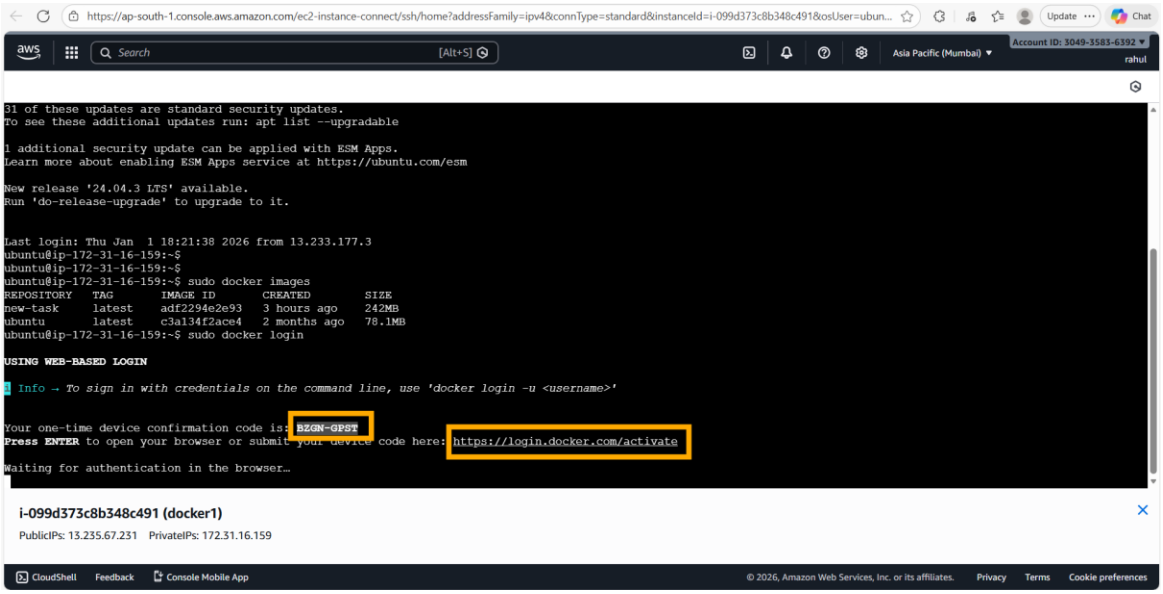
Inside the container on docker2:

```
service apache2 status
service apache2 start
service apache2 status
```

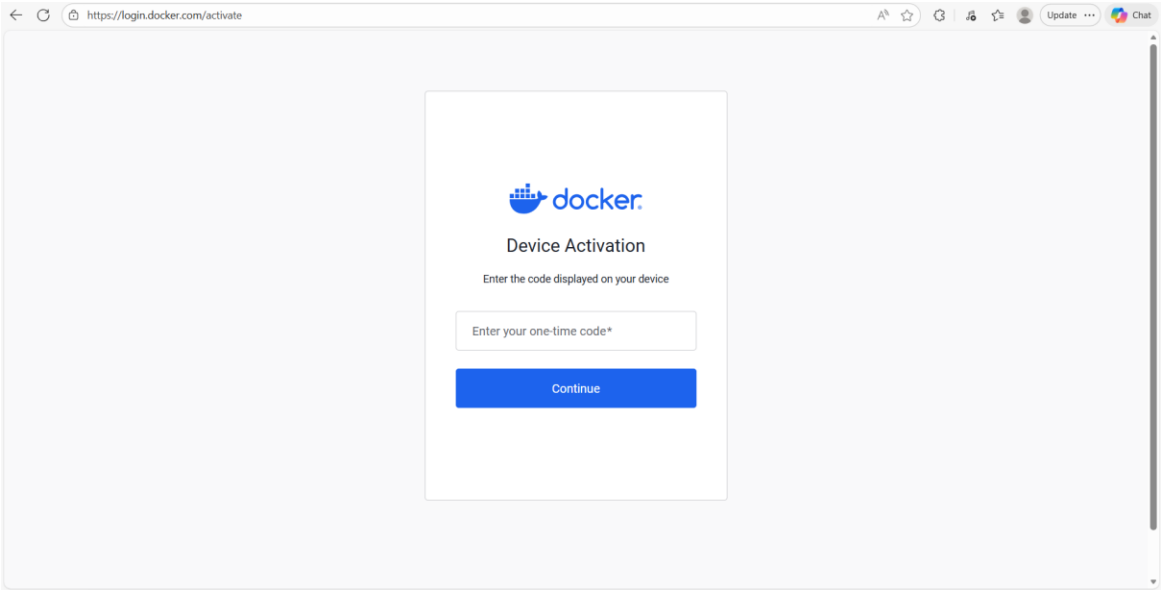
4.2 Screenshots & Comments



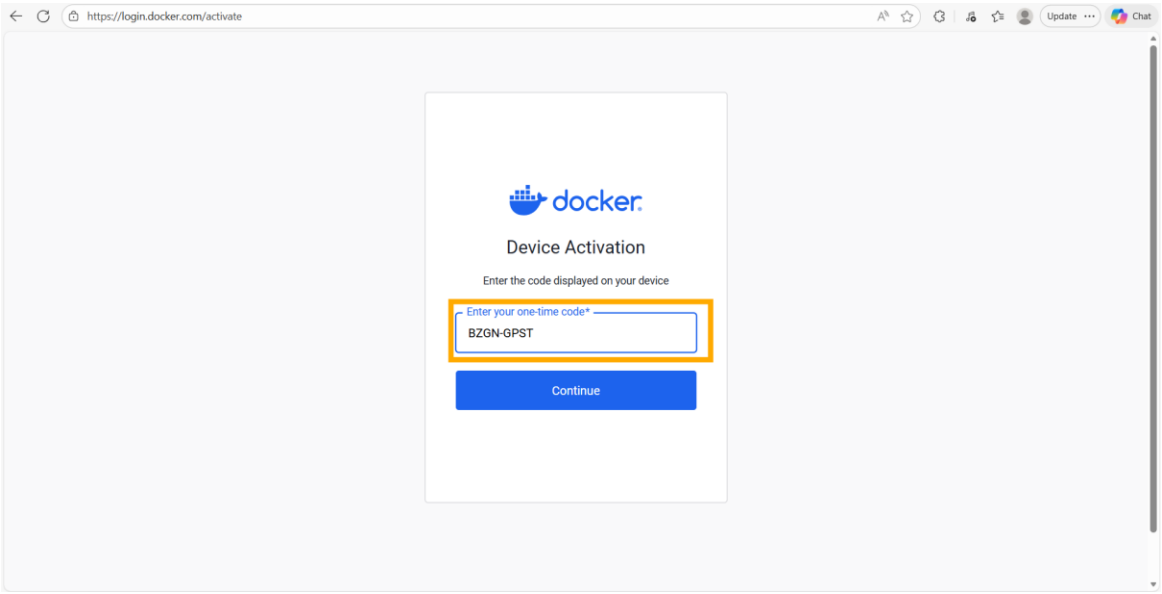
'sudo docker images' on docker1 confirms the 'new-task' image is present (adf2294e2e93, 3 hours old, 242 MB). Running 'sudo docker login' initiates web-based authentication.



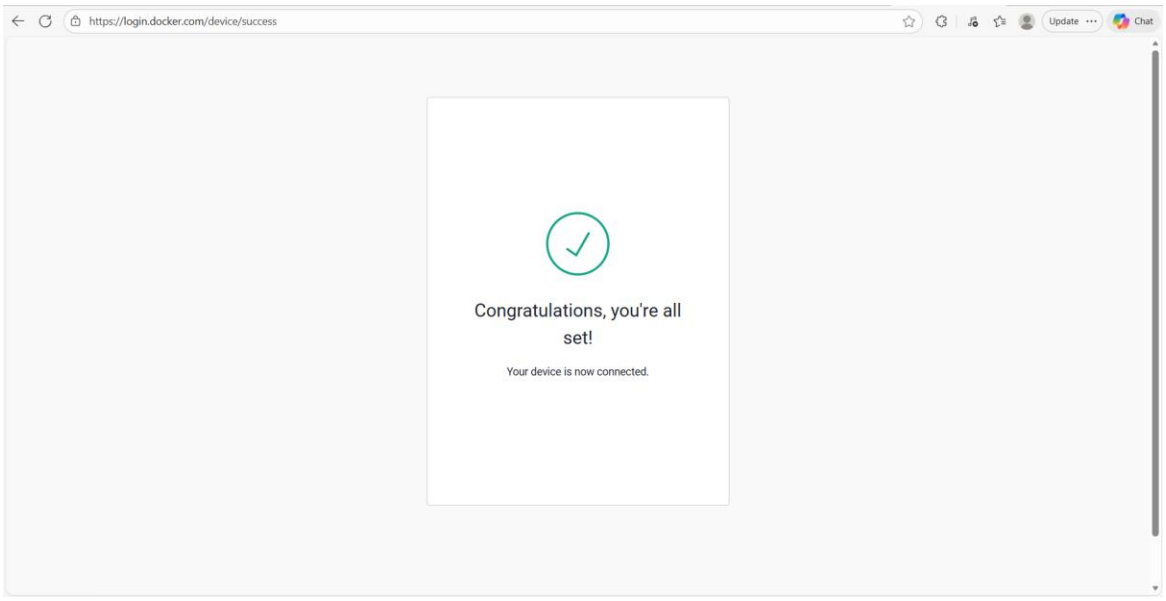
Docker CLI issues a one-time device confirmation code (BZGN-GPST) and provides the activation URL (https://login.docker.com/activate). Waiting for browser authentication to complete.



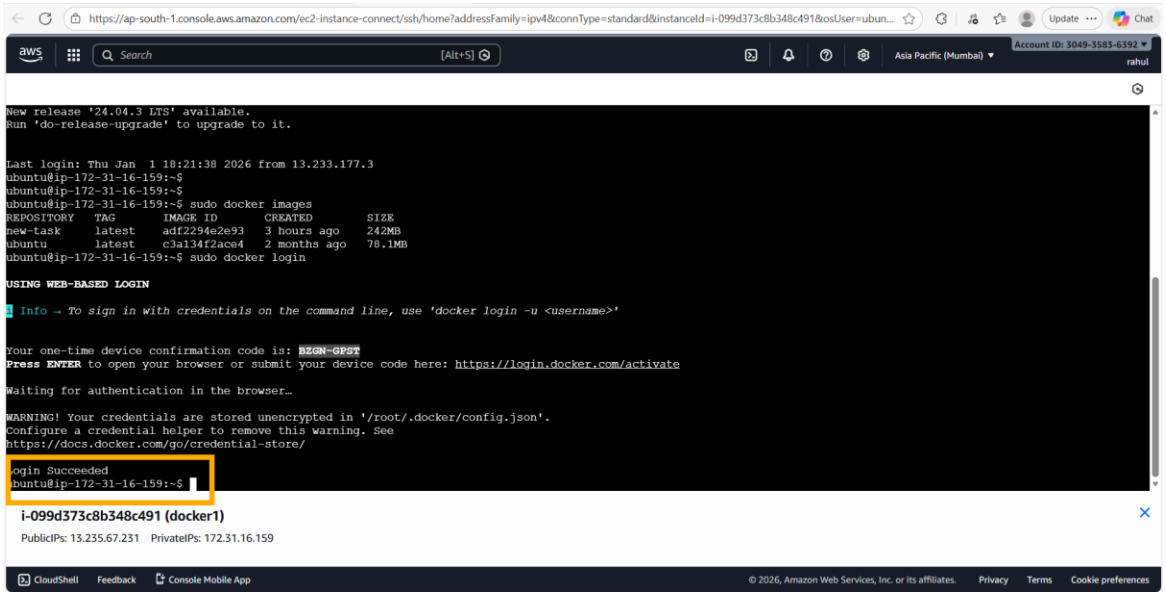
The Docker Hub Device Activation page — the one-time code must be entered here to link the CLI to the Docker Hub account.



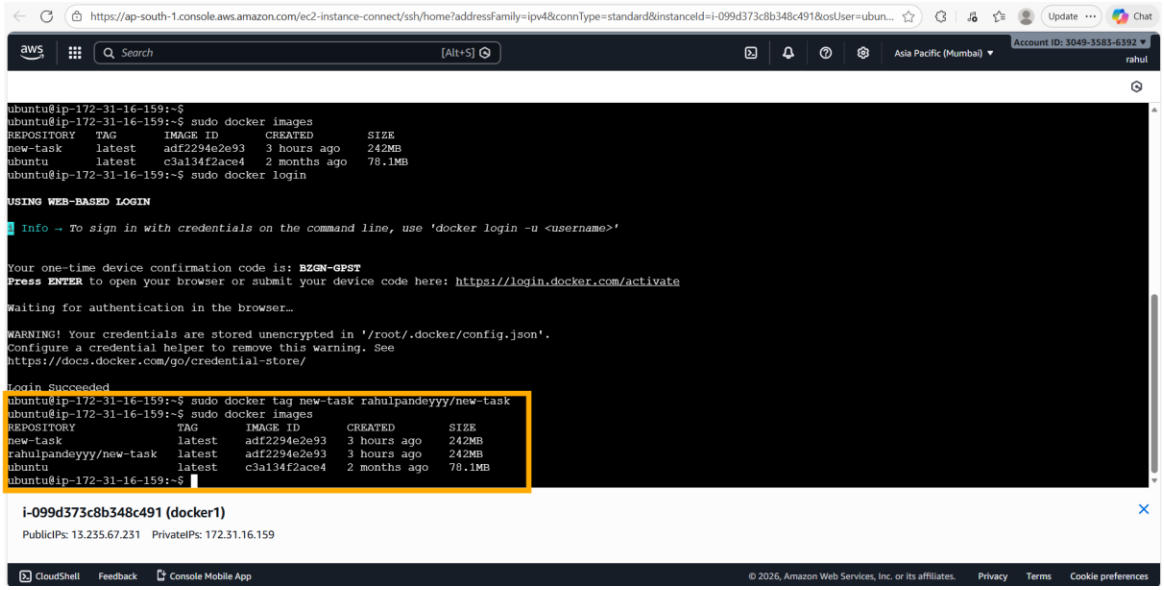
The code 'BZGN-GPST' is entered into the Device Activation form and submitted.



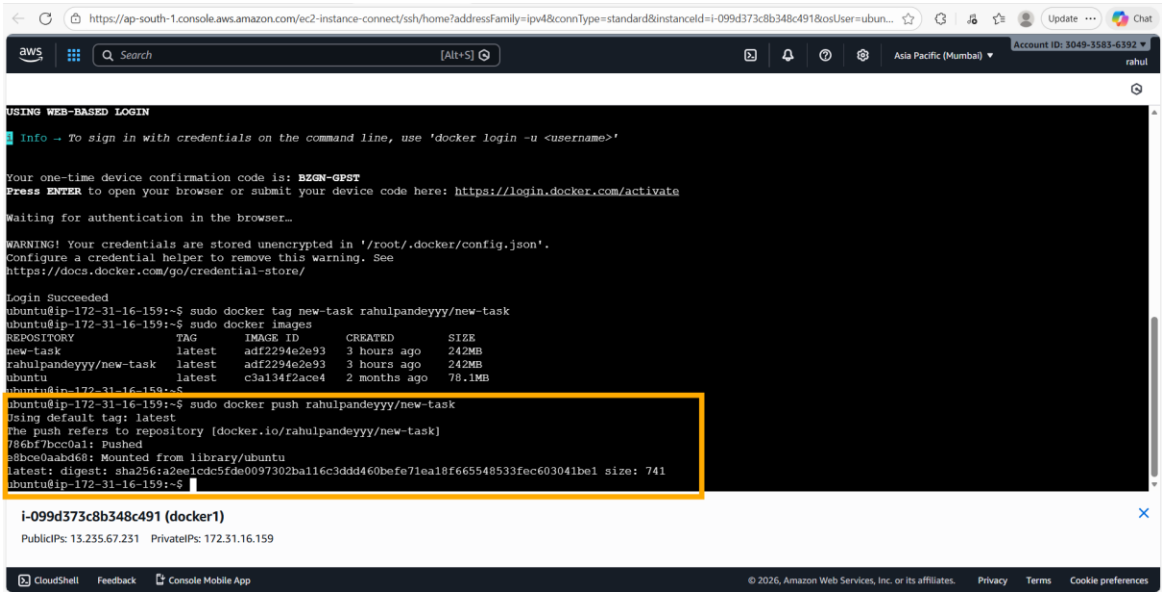
Docker Hub confirms: 'Congratulations, you're all set! Your device is now connected.' — the CLI is now authenticated.



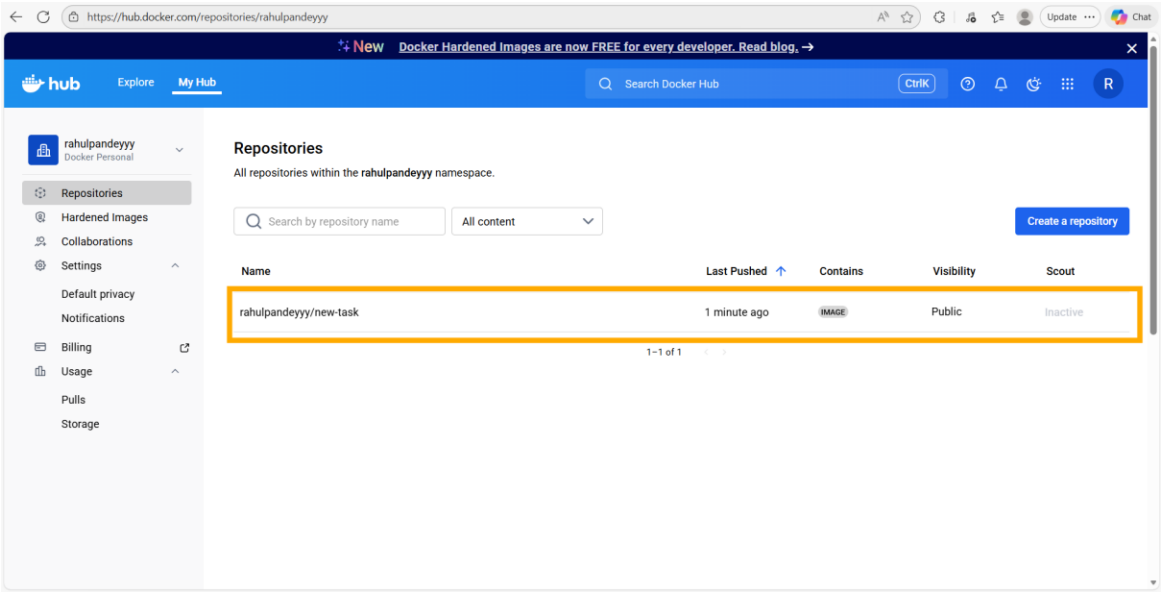
Back in the terminal: 'Login Succeeded' is printed. The credential warning (stored unencrypted) is informational only.



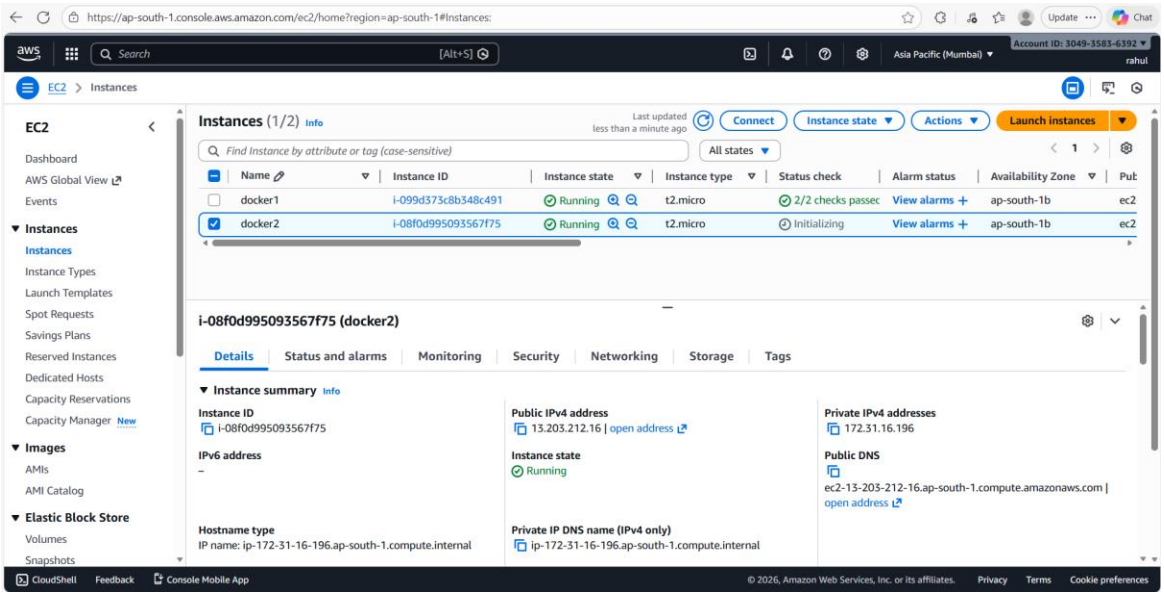
'sudo docker tag new-task rahulpandeyyy/new-task' creates a named tag matching the Docker Hub namespace. 'sudo docker images' shows three entries: new-task, rahulpandeyyy/new-task (same image ID, 242 MB), and ubuntu:latest.



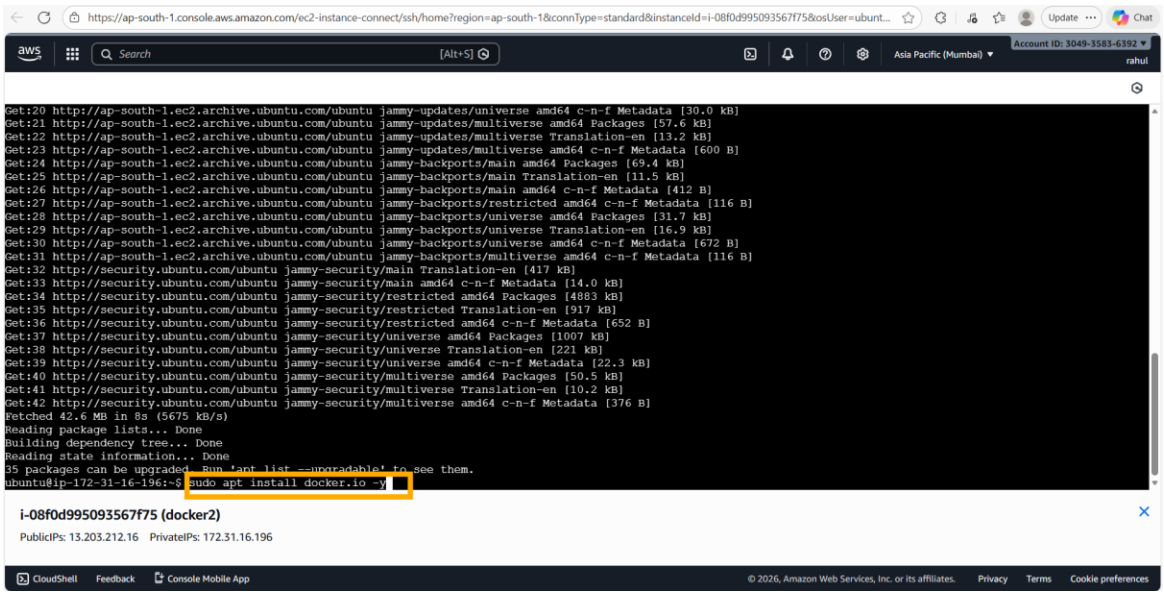
'sudo docker push rahulpandeyyy/new-task' uploads the image layers to Docker Hub. The push completes with digest sha256:a2ee1cdc5fde… and the repository is now public.



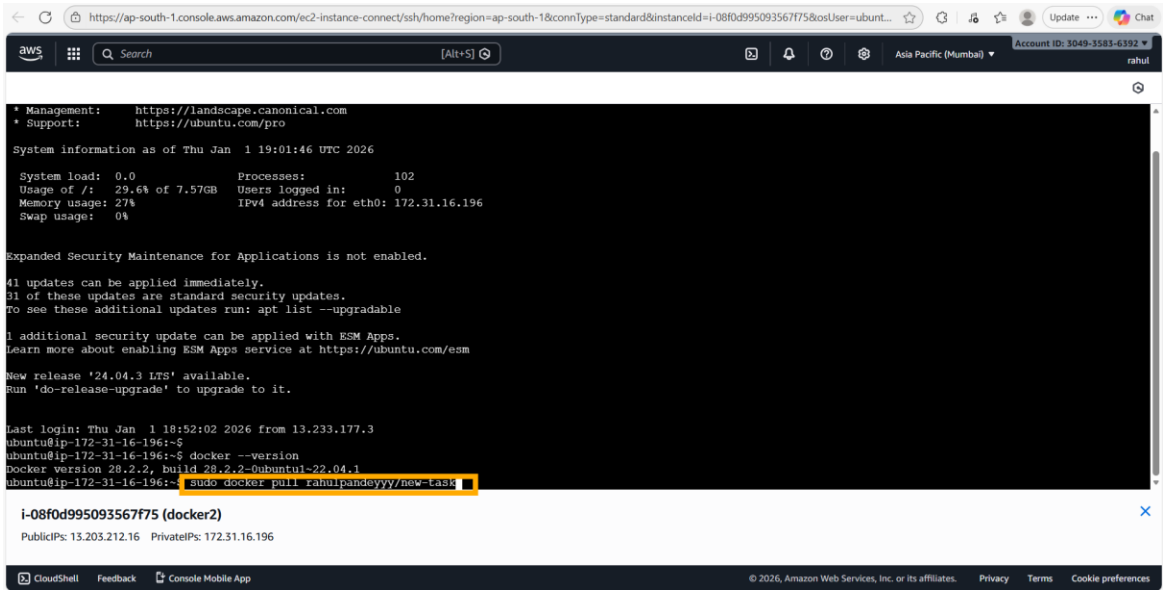
Docker Hub web UI: the repository 'rahulpandeyyy/new-task' is now listed under 'My Hub', marked as a Public IMAGE, last pushed 1 minute ago — confirming the push succeeded.



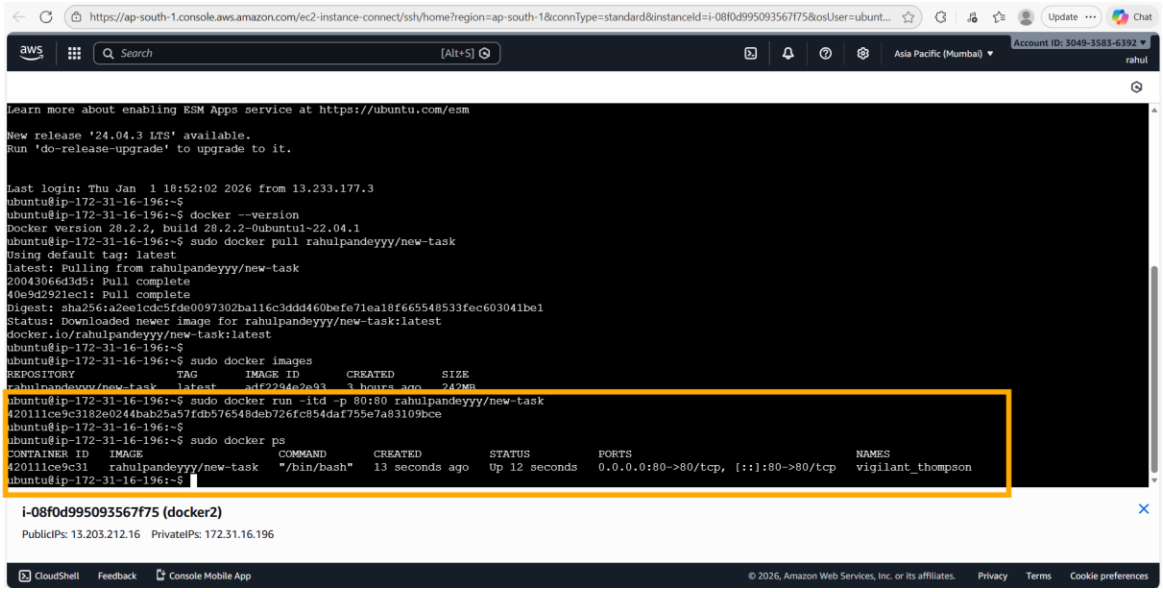
A second EC2 instance named 'docker2' (i-08f0d995093567f75, t2.micro) is launched in the same region (ap-south-1b). Public IP is 13.203.212.16.



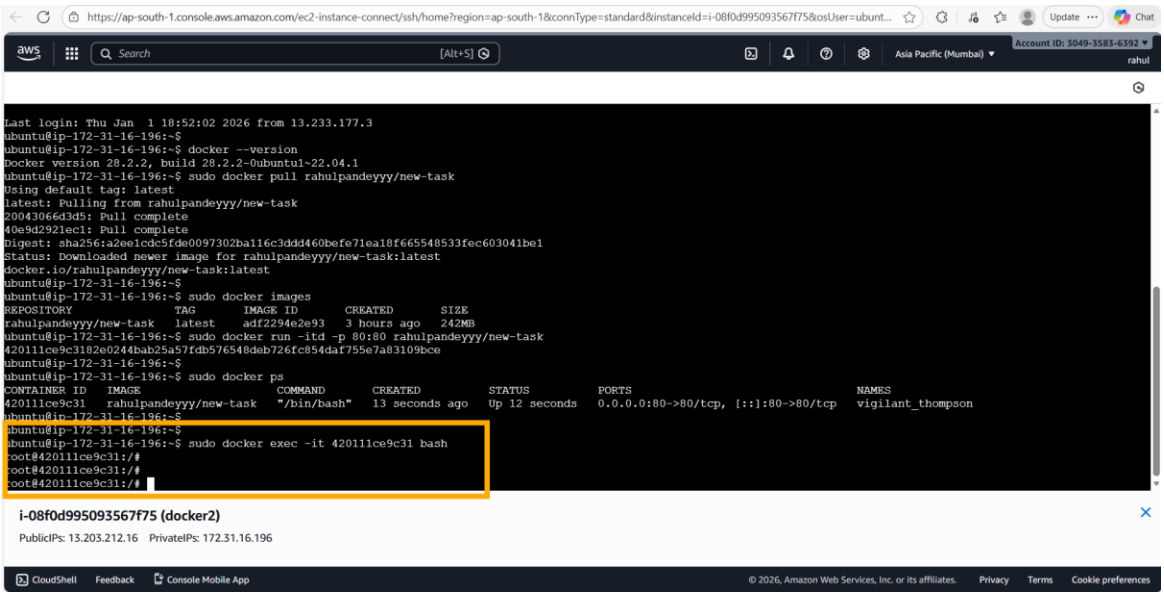
On docker2: 'sudo apt update' refreshes package lists (42.6 MB fetched), followed by 'sudo apt install docker.io -y' to install Docker on this fresh machine.



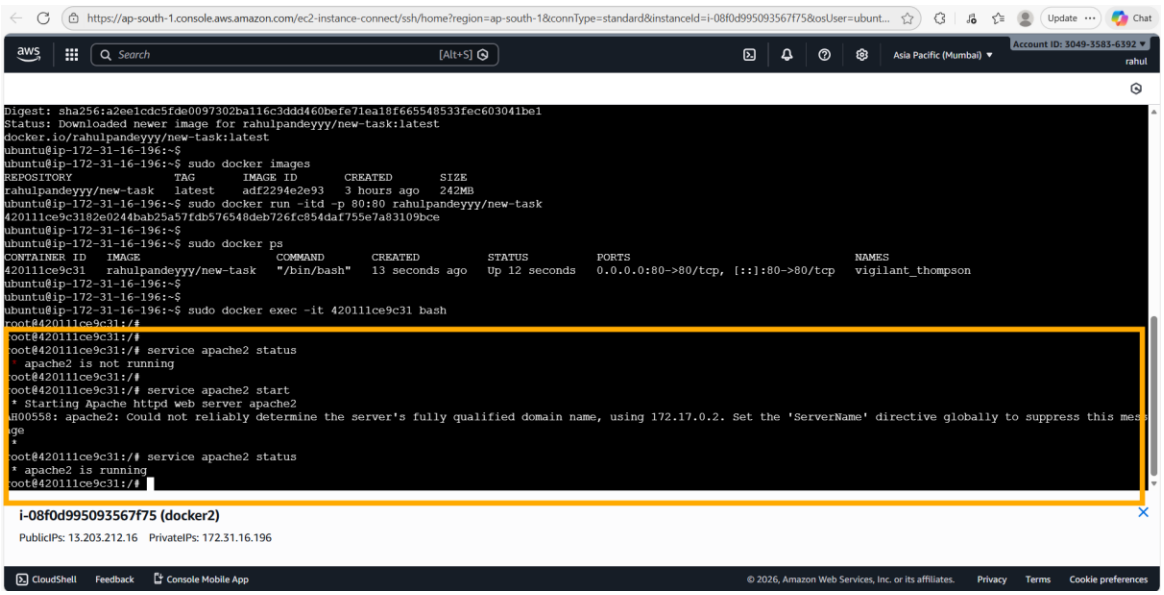
'docker --version' on docker2 confirms Docker 28.2.2. Then 'sudo docker pull rahulpandeyyy/new-task' starts downloading the image from Docker Hub.



The pull completes on docker2 (2 layers fetched, digest sha256:a2ee1cdc5fde…, 242 MB). 'sudo docker images' confirms the image is available locally. 'sudo docker run -itd -p 80:80 rahulpandeyyy/new-task' launches the container, and 'sudo docker ps' shows it is Up with port 80:80 mapped.



'sudo docker exec -it 420111ce9c31 bash' enters the running container on docker2 as root.



Inside the container on docker2: 'service apache2 status' confirms it is not running. 'service apache2 start' starts it. 'service apache2 status' then shows ' apache2 is running' — Apache2 is now serving on port 80.*



Browser verification on docker2's public IP (13.203.212.16): the Apache2 Default Page loads correctly — confirming the Docker Hub image works identically on a completely separate EC2 instance.

5. Assignment 4 — Dockerfile with Auto-Starting Apache2

Tasks: (1) Write a Dockerfile that uses Ubuntu as the base, installs Apache2, and configures it to start automatically when the container starts. (2) Build and run the container. (3) Verify in a browser.

5.1 Dockerfile

```
FROM ubuntu
RUN apt-get update
RUN apt install apache2 -y
ENTRYPOINT apachectl -D FOREGROUND
```

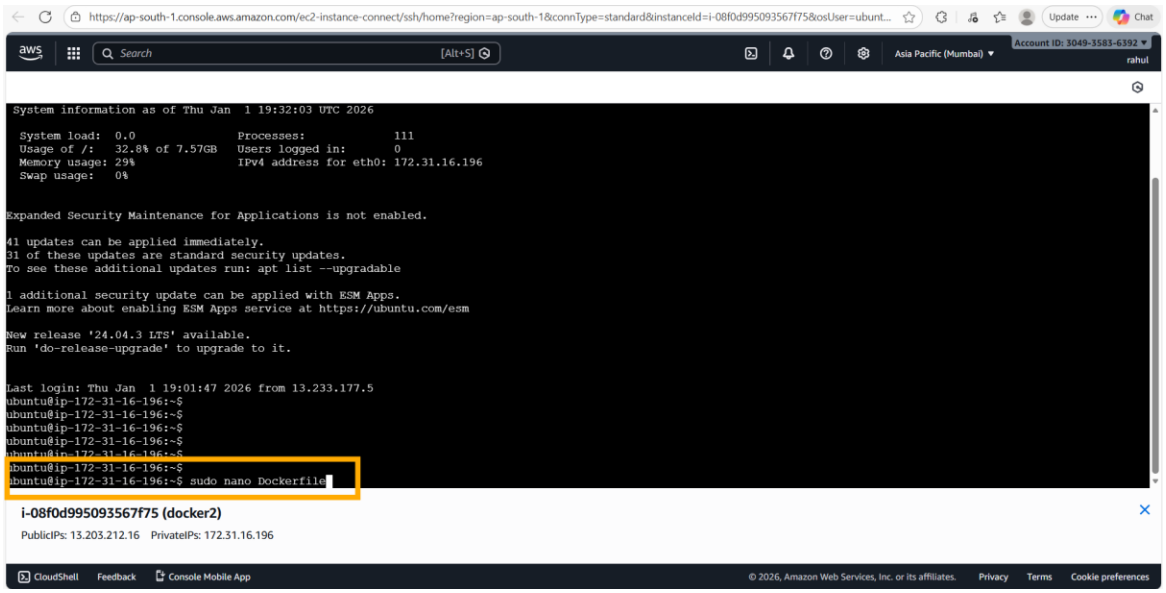
Key design decisions:

- FROM ubuntu — uses the official Ubuntu base image (same as Assignments 1–3).
- RUN apt-get update — refreshes package lists inside the build layer so apt can find apache2.
- RUN apt install apache2 -y — installs Apache2 non-interactively during the image build, baking it into the image layer.
- ENTRYPOINT apachectl -D FOREGROUND — replaces the default container command with apachectl running in foreground mode. Docker containers exit when PID 1 exits; running Apache in the foreground keeps PID 1 alive and Apache serving requests, so no manual 'service apache2 start' is needed after launching the container.

5.2 Deployment Steps

```
sudo nano Dockerfile
# Paste the 4-line Dockerfile above
sudo docker build . -t doc
sudo docker images
sudo docker run -itd -p 99:80 doc
sudo docker ps
```

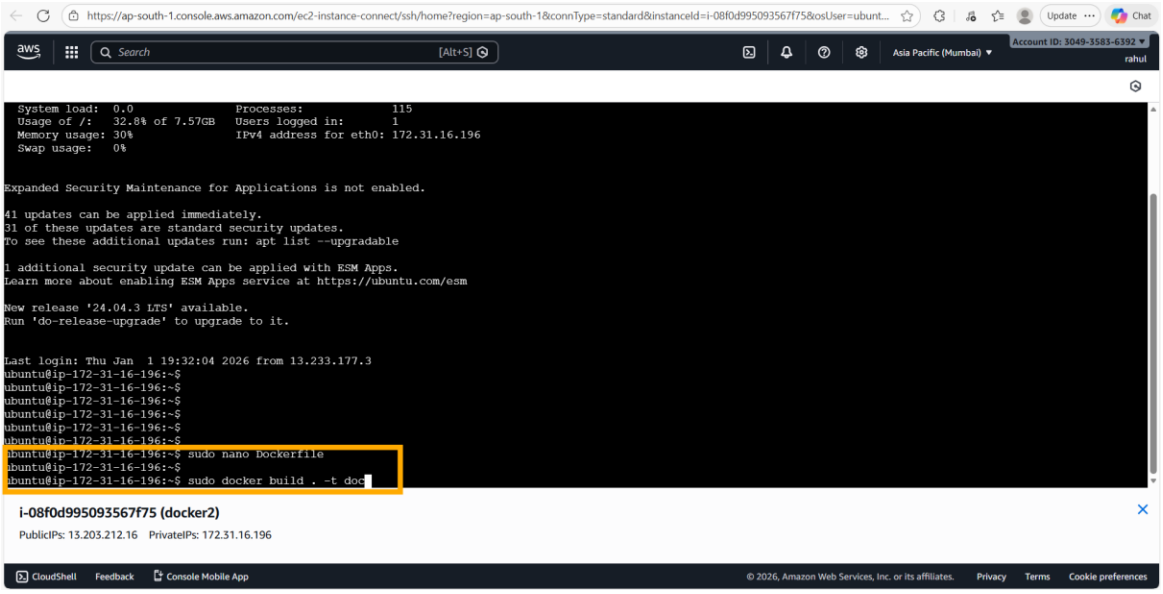
5.3 Screenshots & Comments



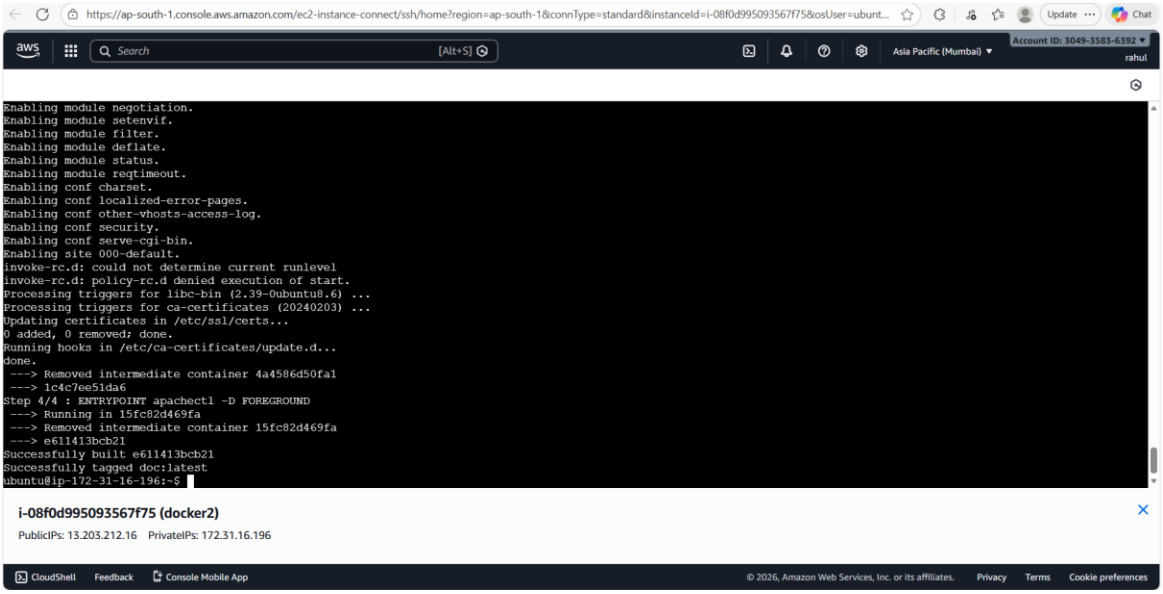
On docker2, running 'sudo nano Dockerfile' to create the Dockerfile — the build context directory is the home directory.



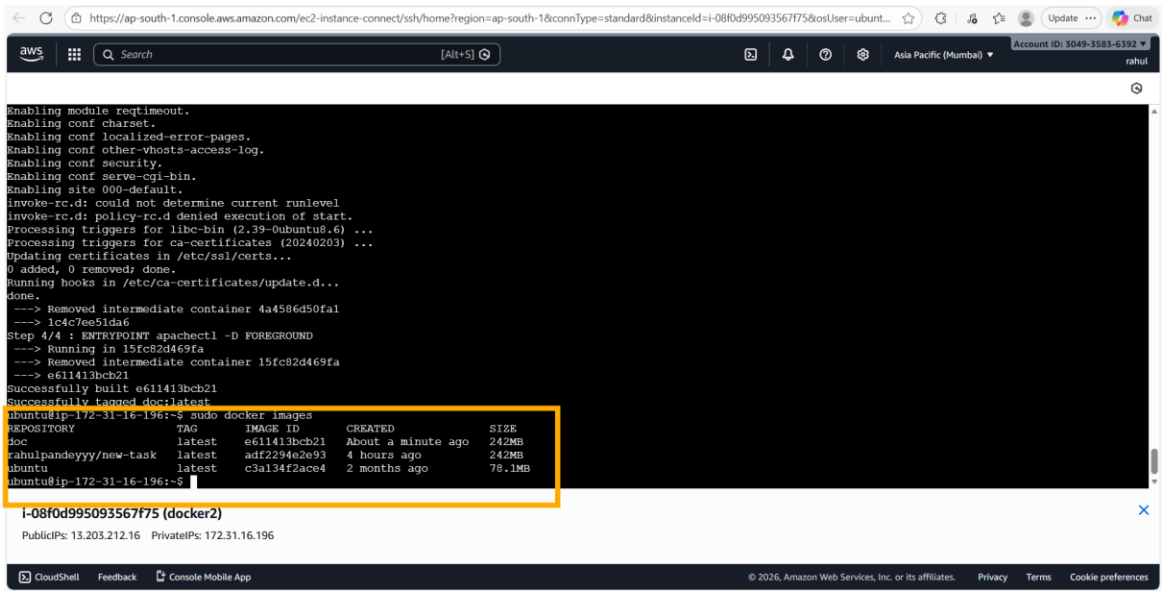
The Dockerfile contents in nano: FROM ubuntu / RUN apt-get update / RUN apt install apache2 -y / ENTRYPOINT apachectl -D FOREGROUND. These 4 lines define a self-contained Apache2 image.



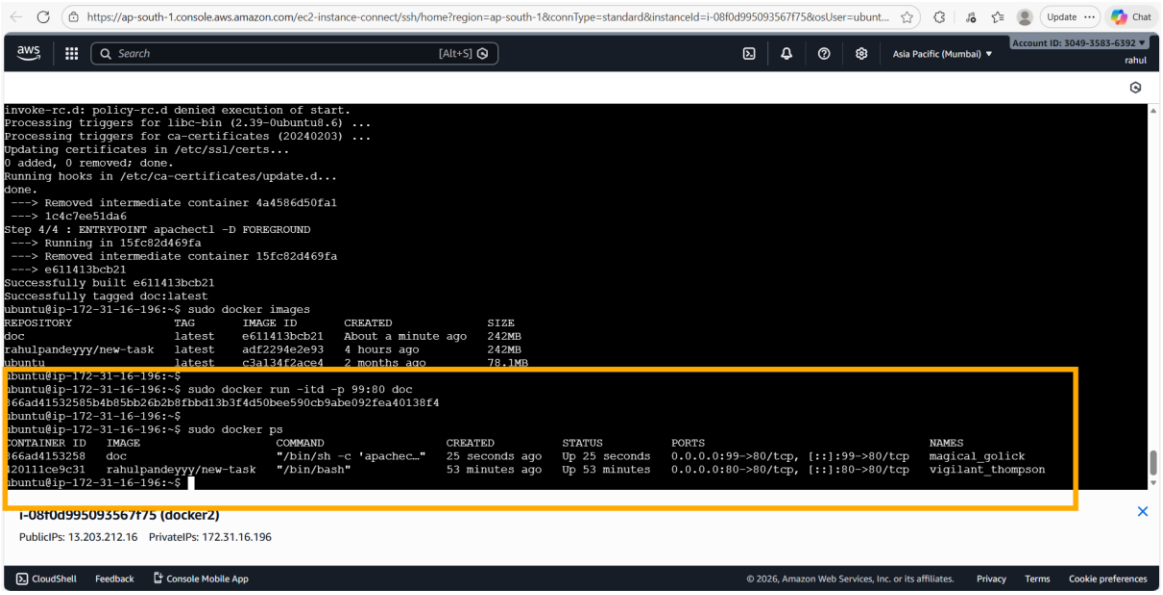
After saving the Dockerfile, running 'sudo docker build . -t doc' to build the image with the tag 'doc'. The build context (15.36 kB) is sent to the Docker daemon.



Docker build output — all 4 steps complete: Step 1/4 FROM ubuntu, Step 2/4 RUN apt-get update, Step 3/4 RUN apt install apache2 -y, Step 4/4 ENTRYPOINT apachectl -D FOREGROUND. 'Successfully built e611413bcb21 / Successfully tagged doc:latest' confirms the image is ready.



'sudo docker images' lists the newly built 'doc:latest' image (e611413bcb21, created about a minute ago, 242 MB) alongside the previous images. 'sudo docker run -itd -p 99:80 doc' launches a container from this image on port 99. 'sudo docker ps' shows both the 'doc' container (port 99:80) and the earlier 'rahulpandeyyy/new-task' container (port 80) running simultaneously.



Docker ps confirms container 866ad4153258 (doc image) is up with COMMAND '/bin/sh -c apachec...' — this is the ENTRYPOINT running apachectl automatically. Port 0.0.0.0:99->80/tcp confirms external access on port 99.



Browser verification: navigating to 13.203.212.16:99 shows the Apache2 Default Page — Apache2 started automatically from the ENTRYPOINT without any manual 'service apache2 start' command.

6. Assignment 5 — Custom HTML Page via Dockerfile ADD

Tasks: (1) Create a custom index.html file. (2) Update the Dockerfile to ADD this file into the container, replacing Apache's default page. (3) Rebuild the image and run a new container on port 120. (4) Verify the custom page in a browser.

6.1 Custom HTML File (index.html)

```
Hi rahul pandey
```

A minimal one-line HTML file. When Apache2 serves /var/www/html/index.html, this replaces the default 'Apache2 Default Page'.

6.2 Updated Dockerfile

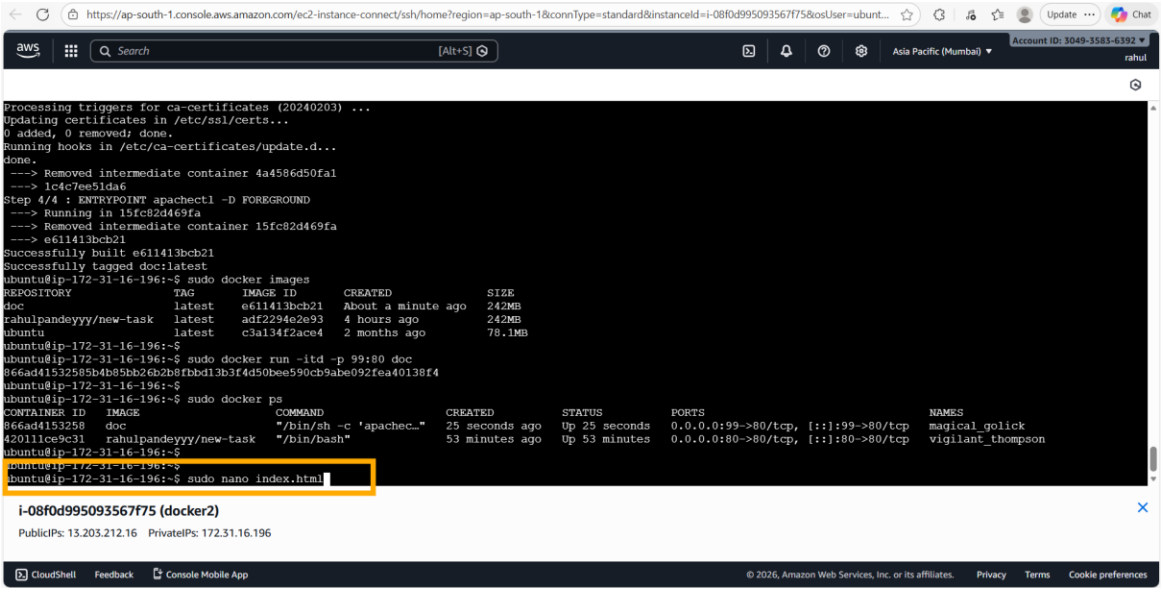
```
FROM ubuntu
RUN apt-get update
RUN apt install apache2 -y
ADD index.html /var/www/html/index.html
ENTRYPOINT apachectl -D FOREGROUND
```

The ADD instruction copies index.html from the build context (the host directory) into /var/www/html/index.html inside the container image at build time, permanently replacing Apache's default welcome page.

6.3 Deployment Steps

```
sudo nano index.html
# Type: Hi rahul pandey
sudo nano Dockerfile
# Add the line: ADD index.html /var/www/html/index.html
sudo docker build . -t work-5
sudo docker images
sudo docker run -itd -p 120:80 work-5
sudo docker ps
```

6.4 Screenshots & Comments



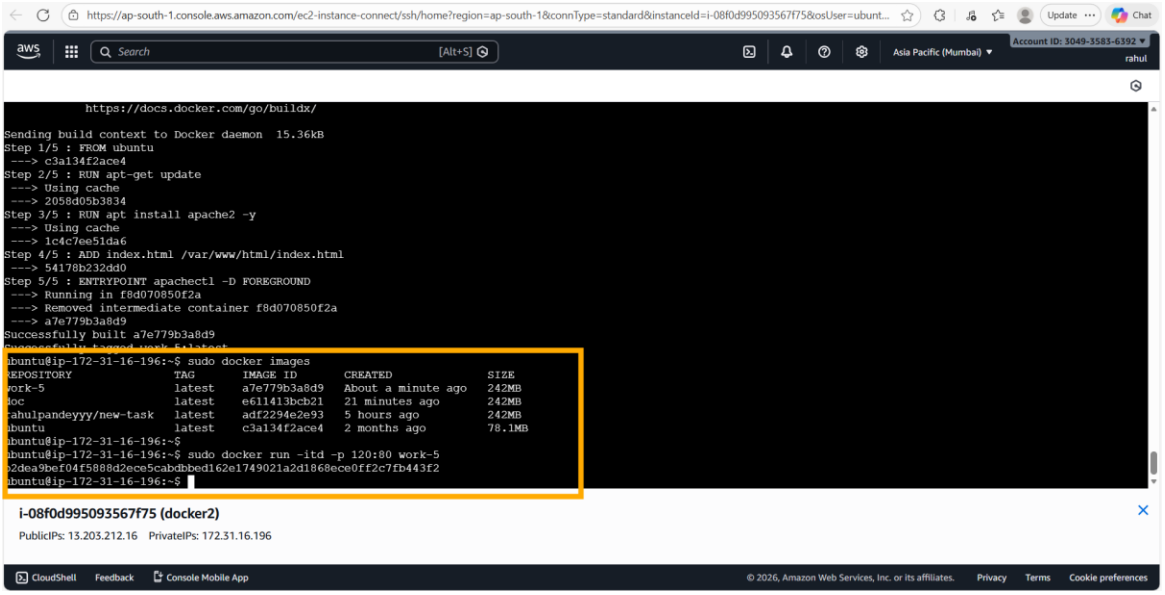
Context shows that the Assignment 4 'doc' container is still running on port 99. Running 'sudo nano index.html' to create the custom HTML file in the current directory (the build context).



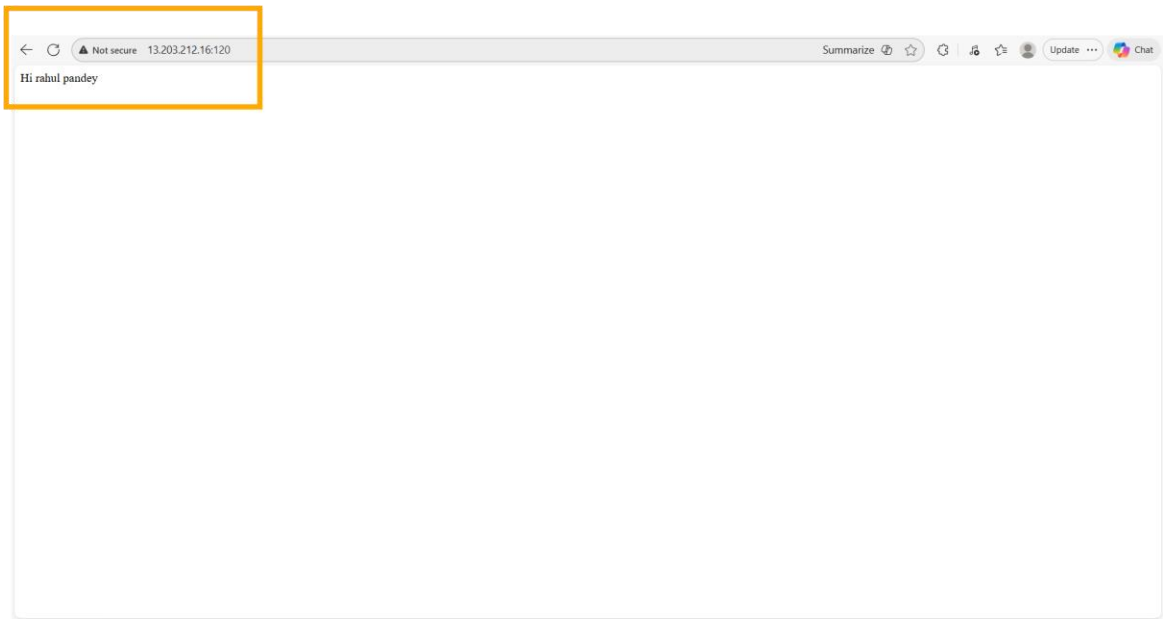
The index.html file in nano contains a single line: 'Hi rahul pandey'. After saving (Ctrl+O → Enter → Ctrl+X), the file is ready to be ADDed into the Dockerfile build.



The updated Dockerfile in nano: an ADD instruction ('ADD index.html /var/www/html/index.html') has been inserted between the apache2 install step and the ENTRYPOINT. This line copies the custom index.html into the container's Apache document root during the build.



'sudo docker build . -t work-5' rebuilds the image (5 steps this time, adding Step 4/5: ADD index.html /var/www/html/index.html). Docker reuses the cached layers from the previous build for steps 1–3 (fast), then executes the new ADD step. 'Successfully tagged work-5:latest'. 'sudo docker images' lists work-5 (a7e779b3a8d9, 242 MB). 'sudo docker run -itd -p 120:80 work-5' launches the container on port 120.



Browser verification: navigating to 13.203.212.16:120 displays 'Hi rahul pandey' — the custom HTML file from the host has successfully replaced Apache's default welcome page inside the container, confirming the ADD instruction works correctly.

7. Consolidated Dockerfiles

The two Dockerfiles developed across Assignments 4 and 5 are reproduced in full below for reference.

7.1 Assignment 4 Dockerfile — Auto-Starting Apache2

```
# Base image: official Ubuntu
FROM ubuntu

# Refresh package lists so apt-get can find apache2
RUN apt-get update

# Install Apache2 non-interactively (-y skips confirmation prompts)
RUN apt install apache2 -y

# Start Apache2 in foreground mode as PID 1 so the container stays alive
ENTRYPOINT apachectl -D FOREGROUND
```

7.2 Assignment 5 Dockerfile — Custom HTML Page

```
# Base image: official Ubuntu
FROM ubuntu

# Refresh package lists
RUN apt-get update

# Install Apache2
RUN apt install apache2 -y

# Copy the custom HTML file into Apache's document root,
# replacing the default Apache2 welcome page
ADD index.html /var/www/html/index.html

# Start Apache2 in foreground mode
ENTRYPOINT apachectl -D FOREGROUND
```

8. Conclusion

Across the five assignments, a complete Docker workflow was demonstrated on AWS EC2:

- Docker was installed from scratch on an Ubuntu EC2 instance, the official Ubuntu image was pulled, and Apache2 was manually installed inside a running container — proving the basic container model works.
- The live container was committed as a new image ('new-task') and immediately relaunched on a different port, showing how docker commit preserves all changes made inside a running container.
- The custom image was tagged, pushed to Docker Hub, and successfully pulled and run on a completely separate EC2 machine — demonstrating Docker Hub as a portable image registry.
- A Dockerfile was written to automate the entire setup: FROM, RUN, and ENTRYPOINT instructions replaced the manual installation steps, producing a self-contained image that starts Apache2 automatically.
- The Dockerfile was extended with an ADD instruction to bake a custom HTML file into the image at build time, replacing Apache's default page — showing how build-time COPY/ADD enables configuration-as-code.

This progression mirrors the path teams follow in practice: from ad-hoc manual container changes, to committed snapshots, to fully automated Dockerfile-driven builds shipped through a registry.